Team TOWEL

Balloon Launch

Tuesday, March 17th, 2026

Tyler Hinkie (EE), Oliver Santiago-Mendez (ED), William Beesley (SE), Ethan Shapero (PL),

Lucas Wang (RE)

Abstract

Numerous factors influence the composition of the atmosphere, and all vary with altitude. The goal of this experiment was to analyze the Earth's atmospheric makeup at various altitudes up to 100,000 feet. The experiment utilized an enclosed payload attached to a high-altitude balloon. This payload housed eleven sensors, including pressure, humidity, temperature, radiation, and GPS, all of which successfully captured data for five individual experiments. These experiments measured humidity, air resistance, pressure, gamma radiation, and temperature differentials (internal vs. external) relative to altitude. After the payload was recovered, the data from both microcomputers were analyzed and, once calibrated, yielded accurate and predictable results.

Table of Contents

Abstract **1**
Introduction and Overview **3**
1.1 Atmospheric Zones 3
1.2 Predictions 7
System Design and Hardware **9**
2.1 Payload Overview 9
2.2 Heating Pad System 12
2.3 Cutdown Mechanism 15
2.4 Buoyancy Calculations 20
Sensors and Calibration **23**
3.1 Sensor Overview and Arduino Configuration 23
3.2 Sensor Calibration 26
3.3 Sensor Related Errors 30
Radio and Tracking Systems **34**
4.1 Yagi Antennas and Foxhunt System 34
4.2 Pico APRS and GPS 36
4.3 Beam-Down System 40
4.4 Radio Related Errors 43
4.5 Predicted vs Actual GPS 45
Data Analysis and Results **46**
5.1 Pressure vs Altitude 47
5.2 Humidity vs. Altitude 50
5.3 Temperature 52
5.4 Ascent/Descent Speed and Air Resistance 54
5.5 Radioactivity vs Altitude 56
Conclusion and Code/Technical Appendix **59**
6.1 Final Conclusion 59
6.2 Code Overview 60
Bibliography **61**

Introduction and Overview

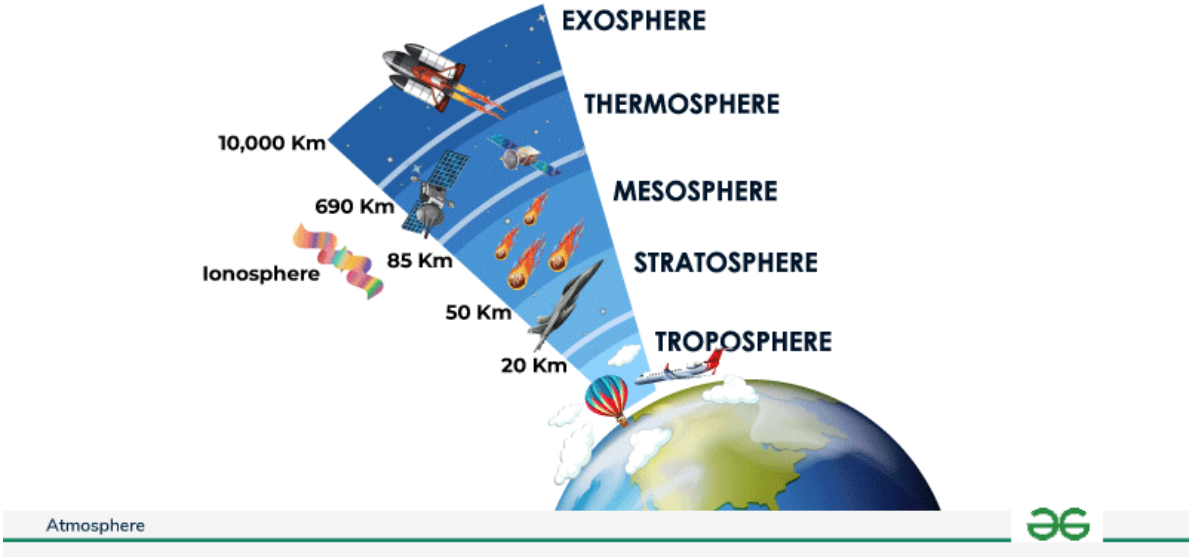


Figure 1.1: Diagram of the Layers of the Atmosphere (GeeksforGeeks, 2025)

1.1 Atmospheric Zones

The atmosphere is broken down into five distinct layers, starting with the troposphere and ending with the exosphere. During the balloon launch, the payload reached a maximum altitude of 100,126ft or 30,520m. This altitude (measured by GPS) means that the payload reached the stratospheric region of the atmosphere, which is between 11km and 50km. To reach this altitude, the payload has to rise through the entire troposphere, which is between 0km and 11km (figure 1.1). As the payload transitions through different altitudes in the stratosphere, the sensors are expected to read a shift in variables such as pressure, temperature, humidity, and radioactivity.

The Troposphere

The troposphere is what most people think of when they say “atmosphere.” It contains almost all of the weather phenomena that exist on Earth, including clouds, rain, hurricanes, and lightning. The troposphere ranges in altitude depending on its proximity to the poles; as the proximity increases, the height of the troposphere decreases. At the poles, the troposphere is just 9km high, while at the equator, it is up to 20km.

The payload was launched from [redacted], which is at ~38.5º north. While no one equation can determine the exact height of the troposphere at any given location, scientists often use the empirical equation $h(\varphi) = 16.5 - 0.15|\varphi| + 0.0005\varphi^2$ to approximate. By plugging in 38.5º north to the approximation equation, it can be determined that the height of the troposphere was ~ 37,500 feet or 11.4km.

The troposphere also contains almost 80% of all air on Earth by mass, while only taking up 10% of the vertical altitude. This fact is due to the barometric law, which states that as altitude increases, the pressure decreases exponentially. At the top of the troposphere, the payload is expected to read around 20kpa.

The temperature of the air in the troposphere also decreases with altitude. At the borderline between the troposphere and the stratosphere, the temperature will be at its lowest. This is because while the troposphere is heated from the ground, the stratosphere is heated via UV radiation. At the borderline between the two, the payload would be farthest from both heat sources and therefore experience the coldest temperature, which is expected to be around -60º C

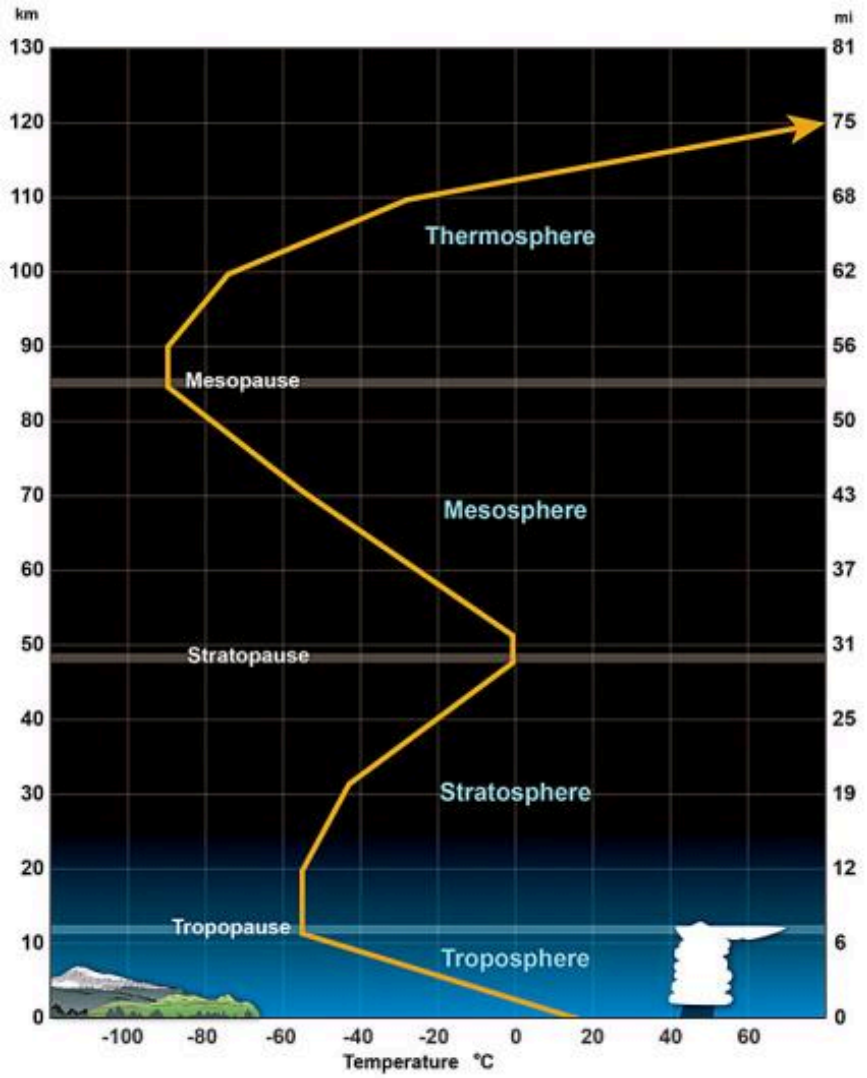


Figure 1.2: Altitude vs Temperature Through the Atmosphere (NOAA, 2026)

The Stratosphere

The stratosphere is the second layer of the atmosphere, sitting right above the troposphere. It extends from around 11 km up to about 50 km (31 mi) above the Earth's surface. This layer holds about 19% of the atmosphere's gases but has almost no water vapor.

Unlike the troposphere, temperature in the stratosphere increases with altitude. This is because the ozone layer exists within the stratosphere; UV radiation from the sun gets absorbed

during ozone formation, which releases heat. Temperatures climb from around -60°F (-51°C) at the tropopause (border between troposphere and stratosphere) up to about 5°F (-15°C) near the top of the stratosphere.

Since warmer air sits on top of cooler air in this layer, there's no convection, meaning the gases don't mix vertically the way they do in the troposphere. It is for this reason that cumulonimbus clouds flatten out into anvil shapes at the base of the stratosphere because they physically can't push through the temperature inversion above them.

The payload spent most of its flight in the stratosphere, climbing from the tropopause all the way up to a peak of 30.5 km (over halfway through the layer). At peak altitude, the calibrated pressure had dropped to just 8.6 mbar, and the outside temperature had warmed back up from its minimum of -64°C at the tropopause to around -20°C, which lines up with the expected stratospheric temperature profile.

Upper Atmosphere

Above the stratosphere sit three more layers that the payload did not reach. The mesosphere extends from 50 km to 85 km (31-53 mi). This layer is where the atmosphere becomes thick enough to burn up incoming meteors. Temperatures in this layer drop with altitude, reaching as low as -120°C near the top, making it one of the coldest places on Earth.

Above that is the thermosphere, stretching from 85 km up to around 600 km (375 mi). Despite temperatures reaching as high as 2,000°C due to absorption of X-ray and UV radiation, the air is so thin that it would still feel cold. The International Space Station orbits within this layer, and it's where auroras occur.

The exosphere, which is the outermost layer, goes from around 600 km out to roughly 10,000 km. At these altitudes, the atmosphere is so thin that individual atoms and molecules travel rarely, or even collide, and the layer gradually fades into full space.

The payload's peak altitude of 30.5 km put it just over halfway through the stratosphere witch is still well below the 50 km stratopause boundary. Reaching the mesosphere would have required the balloon to climb an additional 20 km, and reaching the thermosphere would have meant a further 35 km beyond that. These upper layers are inaccessible to weather balloons and are typically only reached by sounding rockets and orbital spacecraft.

1.2 Predictions

In order to design the experiments and sensor suite properly, the team had to have a good idea of what conditions the payload would actually experience before it ever left the ground. The balloon used was one known to have reliably reached 100,000 ft, and was filled to approximately 7.5 lbs of buoyancy. The target altitude was set at roughly 100,000 ft, and all sensor ranges, battery life, insulation, and heater thresholds were designed around that number.

Using pressure models, it was found that at 100,000 ft the outside pressure would drop to approximately 10 mbar, which is 1% of sea level pressure, and that the coldest temperatures would be around -60°C near the tropopause. These predictions helped determine components and testing conditions. When doing tests, the payload was placed in simulated conditions similar to these predictions.

Flight path predictions were generated using SondeHub (sondehub.org), which takes real-time wind forecast data and estimates the full trajectory, burst point, and landing zone based

on payload weight and balloon type. In this case, the predictions showed that the balloon was likely to land near Patterson, CA. This data was used to pick a launch day that would keep the payload in a recoverable area and away from terrain like mountains or bodies of water.

The flight also had to comply with the FAA, which governs weather balloons in the US. The team supervisor filed a NOTAM and notified the nearest ATC facility before launch. The payload was kept under the 4-pound limit at a final weight of 3.7 lbs, was below maximum density, and included the required cutdown mechanism.

System Design and Hardware

2.1 Payload Overview

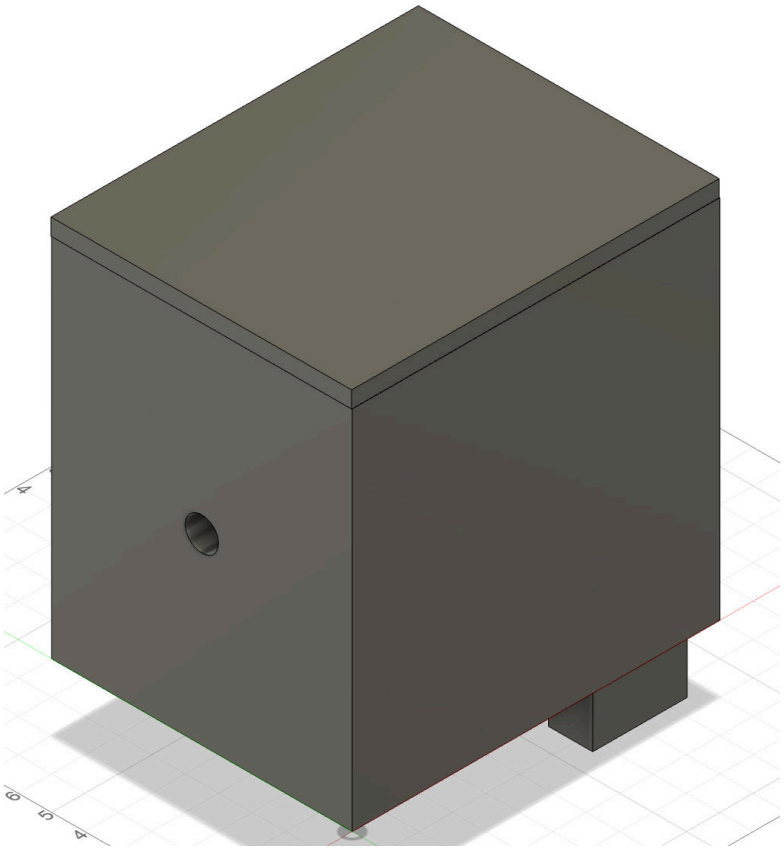


Figure 2.1.1: CAD of our Payload

The payload was developed to contain all the electronic components safe throughout the flight while enduring harsh environmental factors such as temperature as low as -60°C and mechanical stress during both ascent and descent. For this payload, we build the casing our of styrofoam, which is a light material that also has insulating properties, making it an essential factor for our payload staying warm throughout the flight. On the exterior, there is a circular opening where we passed a temperature sensor and a humidity sensor. On the inside, there is a

three-tiered rack system that we 3D printed to hold components in a way that stacks them, helping with wire management.

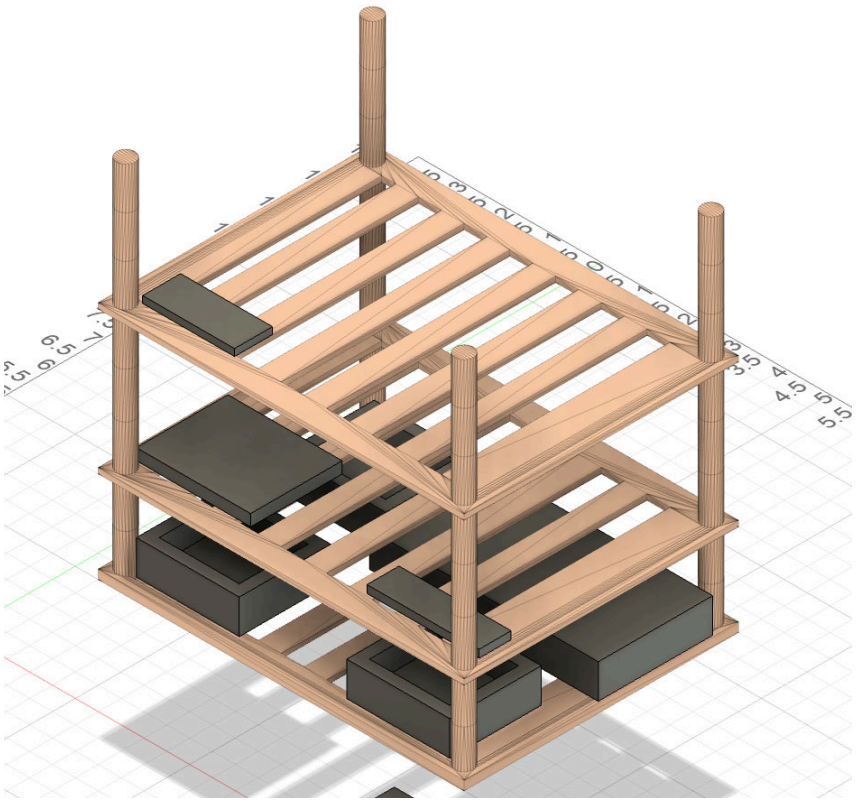


Figure 2.1.2: CAD of our Rack System

Bottom Layer

The Bottom layer stored the batteries, the power bank, and our beam down component. The rationale behind placing the heaviest components in the bottom layer was to reduce the center of gravity of the payload, keeping it stable throughout the flight, and adding less stress on the rack compared to if we placed the components on the top layer.

Middle Layer

The middle layer contains the Arduino, the Teensy, another set of batteries, and the heating pad. The rationale behind the inclusion of the heating pad was that the components had

to be maintained at a minimum operating temperature. The inclusion of the main computing components in the middle layer was aimed at protecting the components from shock, in case something were to go wrong with our parachutes.

Top Layer

The top layer holds the fox hunt tracker, the Pico, and the cutdown system. The cutdown system especially benefitted by being at the top because it had the task of cutting the balloon connection.

Component Selection

Each component was chosen based on the need for reliability, weight, and compatibility with the rest of our circuits. The use of the Teensy was redundant in case the Arduino did not log our data, but the Arduino was used as the main data logging controller based on its widespread support. The heating pad was chosen based on the need to keep electronics running in the cold temperatures of the atmosphere, while the styrofoam housing kept the inside insulated throughout the flight. Our team's GoPro was put on the bottom of the box so that neither the box or the balloon are in the way of the camera and we can get a clean view as it rises up. Additionally, we bought batteries that are cold resistant for extra redundancy.

Payload Errors

Despite the success of the data collection, the payload experienced a significant issue during the descent and landing phase. The primary cause was a parachute deployment failure, which resulted from a calibration error in our code. Because the altitude readings were not accurate, the cutdown system never went off. This caused the payload to enter an uncontrolled free-fall from approximately 100,000 ft. The payload landed on the ground at an estimated 40 mph. The impact caused the outer styrofoam housing to break apart. Upon impact, the inner rack

system was destroyed and our SD cards were ejected from their slots, which made it stop recording data.

2.2 Heating Pad System

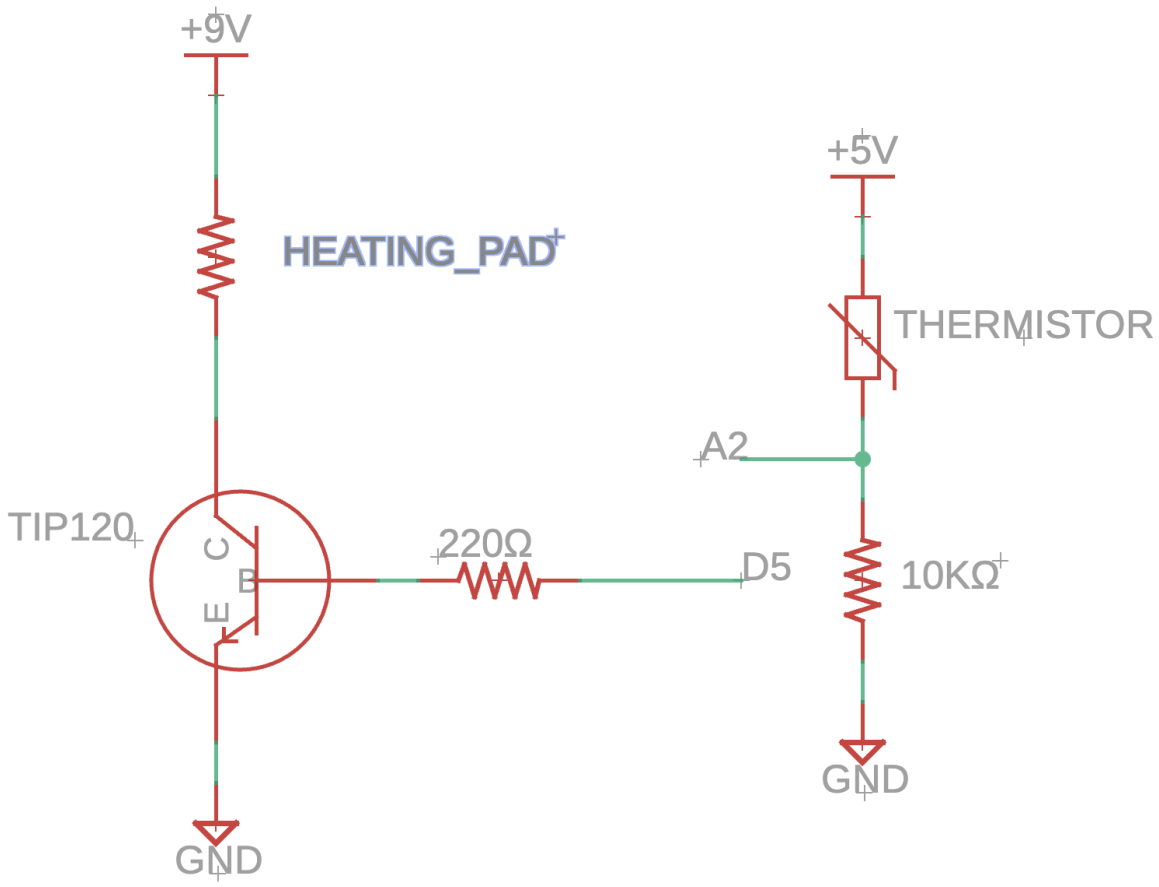


Figure 2.2.1: Heating Pad Schematic, Pins D5 and A2 of the Arduino Labeled

The transistor is a TIP120, an NPN Darlington Power Transistor, configured as a low-side switch, controlled by the Arduino, and an NTC thermoresistor working as a temperature sensor. The thermoresistor forms a voltage divider with a fixed 10kΩ resistor. As the temperature falls, the thermistor's resistance increases, lowering the voltage at analog pin A2, which the Arduino samples to determine the current temperature. When the temperature drops below the

set threshold of 10°F, the Arduino sets digital pin D5 HIGH. This outputs voltage to the base of the transistor through a current-limiting resistor. In this state, the transistor conducts current from collector to emitter, completing the heating pad circuit to ground. This closed circuit allows current to flow through the heating pad, activating it and increasing the internal temperature. Once the temperature rises to 15°F, the Arduino sets D5 to LOW, removing base current and opening the heating pad circuit, interrupting current flow and turning the heating pad off.

Performance and Effectiveness:

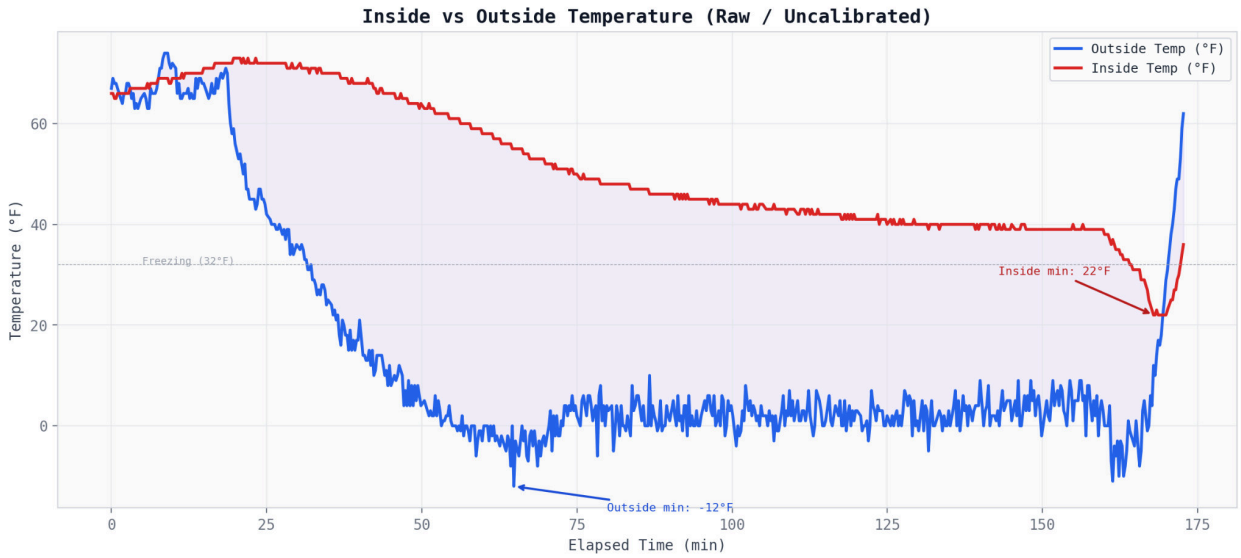


Figure 2.2.2: Raw Uncalibrated Flight Temperature Data Over Time

The temperature data retrieved from the flight revealed that the sensor the Arduino was using did not reflect the actual internal temperature of the payload. The uncalibrated sensor recorded a minimum of 22°F, never crossing the 10°F activation threshold programmed into the heating system. As a result, the Arduino did not trigger the heating pad circuit until the end of the payload's flight, limiting the heating pad to single activation rather than the sustained thermal regulations the system was intended to provide.

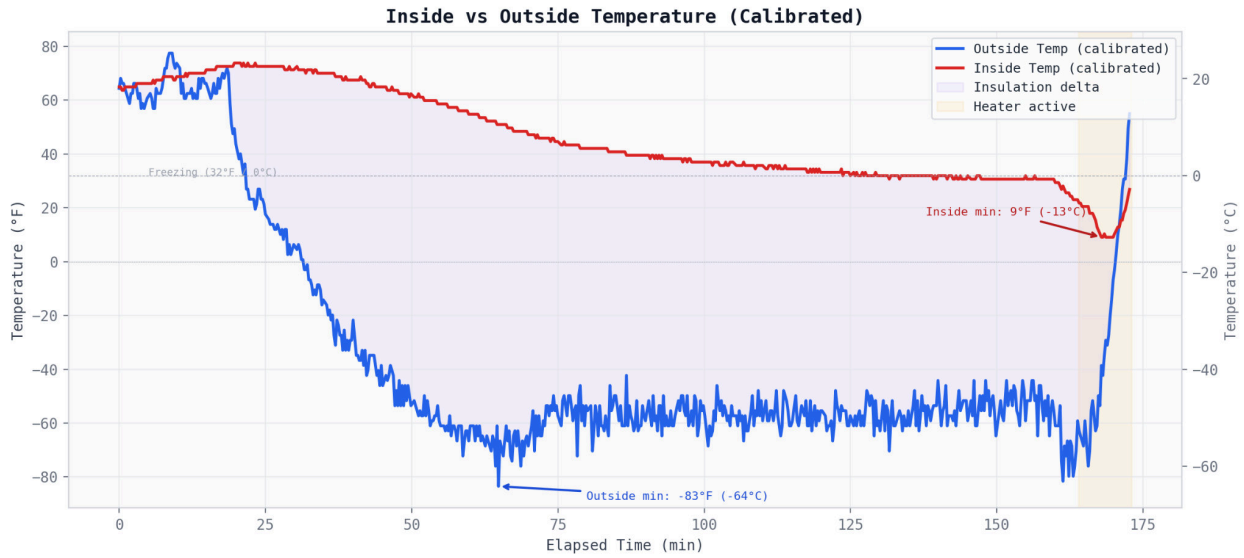


Figure 2.2.3: Calibrated Flight Temperature Data Over Time

After calibrating the data post-flight, the true internal minimum temperature was 9°F, which means the inside of the payload did get cold enough to trigger the activation threshold of the heating system. The calibration shows that the sensor reading was 13°F warmer than the actual temperature the whole flight, which is why the heating pad never activated when it should have.

Because the Arduino relies on the thermoresistor's voltage readings to make all switching decisions, an uncalibrated sensor shifts what the Arduino perceives as the activation threshold. The heating system itself responded correctly to the data received, the information that it received led it to not activate at the right intervals. Had the sensor been calibrated before the launch, the internal temperature readings would have crossed the 10°F threshold much earlier, causing the heater to cycle on and off between the 10°F and 15°F thresholds as designed.

The heating pad circuit performed as intended; the lack of pre-flight sensor calibration led to late activation of the heating pad. The core issue was not a failure of the circuit itself, but a failure of the thermoresistor readings. For future flights, calibrating the thermoresistor against a

known reference temperature before launch would ensure the heating system functions as intended throughout the flight.

2.3 Cutdown Mechanism

One of the most important roles in sending anything to space is planning how it might return. Most of the time, this involves two components: (a) switching from ascent to descent and (b) calculating how to recover the equipment. In this project, the electrical engineer was in charge of component (a).

The payload was directly attached to the balloon via a tether, so switching from ascent to descent required severing the bond between payload and balloon. There were a few viable options, including a razor blade to cut the string, but the team decided to burn the string with nichrome wire instead. It was more predictable and far less subject to hard-to-predict air movements in the stratosphere.

With this Nichrome process, the first step was figuring out how to store energy and discharge it quickly through the wire. Thus, an obvious option emerged: capacitors. However, not just any old capacitor would work. The circuit required ultracapacitors, which have higher capacitance to reduce discharge time and increase the amount of charge stored. Capacitors also discharge more quickly than batteries, allowing for the rapid power spike required to make a nichrome wire red-hot.

Then a switch was needed to control exactly when the capacitors discharged through the nichrome. It quickly became clear that an electrical switch would be far easier to control (and set up) than a physical one. After using transistors previously, MOSFETs were found to be far more

effective for this specific purpose because MOSFETs are controlled by voltage instead of current. The reason this is important in this scenario is that the cut-down mechanism was being triggered by an Arduino, which can handle very limited current. After sourcing some MOSFETs that needed far less voltage at the gate to connect the drain and source pin, the rest of the circuit was pretty self-explanatory. A battery or power supply in parallel to the capacitor bank was needed to charge them, and the resistors were there to prevent accidental triggering during Arduino start and to keep the input current from damaging the Arduino.

The biggest problem was securing Nichrome to the solder board. After trying some solder techniques to lock the nichrome in place, anything with solder turned out to be too inconsistent a connection with the nichrome for consistent cut-down results. Thus, a new approach was needed: crimping. This created physical connections between regular cables and nichrome, which allowed for a consistent connection at varying temperatures and pressures with far less (nearly zero) chance of getting dislodged. This was confirmed through extensive testing, yanking as hard as possible on both sides of the crimped connection.

Once the design and all components were understood and mapped out, all that was left was to test. Here is the final circuit design:

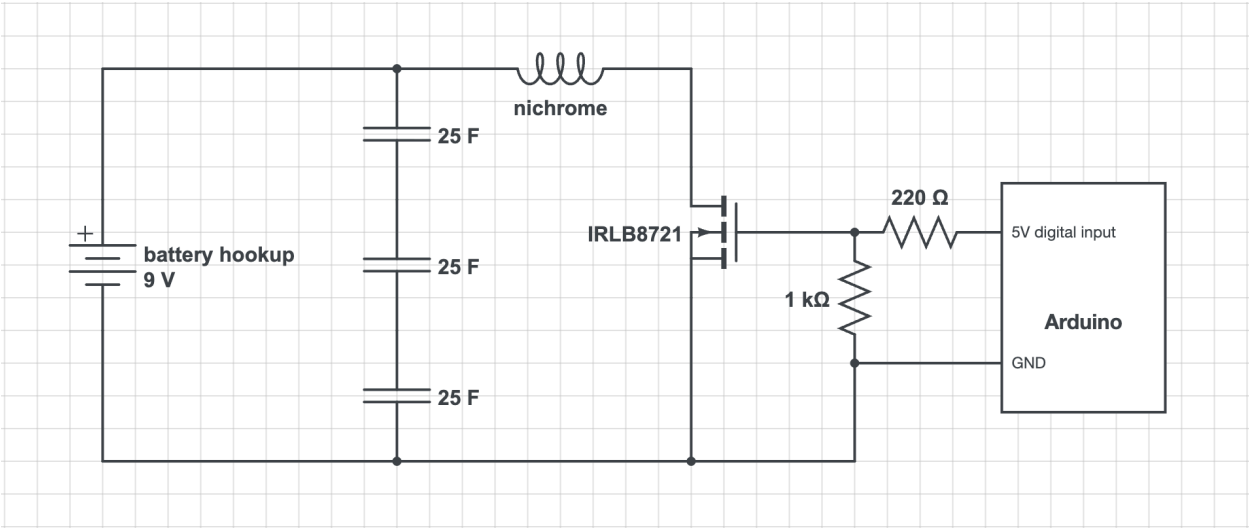


Figure 2.3.1: Final Circuit for Cut-down Mechanism.

In preliminary testing, the exact amount of power and energy needed to heat the wire properly had to be found, and so tests were conducted with a bunch of different voltage amounts to find the proper amount of charge/ultracapacitors needed for the circuit.

Current (A)	Voltage (V)	Power (W)	time (s)
0.85	2.7	2.295	9.83
0.75	3.3	2.475	9.66
0.88	3.4	2.992	4.57
1.2	3	3.6	3.12
1.8	2	3.6	6.88
1.5	3.5	5.25	1.56

1.8	3.6	6.48	1.23

Figure 2.3.2: Power and Cut-down Time Tests

In the interest of extra caution, the team decided to aim for 7W of power and thus used 3 ultracapacitors. One interesting fact about ultracapacitors is their high capacitance and thus ability to hold onto charge before being released. Knowing that it would be at least an hour during ascent from launch to cut-down, the capacitors were tested to determine if they naturally discharged over time.

t (mins)	V (V)	Q (C)	E (J)
0	8.3	69.16666667	287.0416667
5	8.2	68.33333333	280.1666667
10	8.1	67.5	273.375
15	8.1	67.5	273.375
20	8	66.66666667	266.6666667
25	8	66.66666667	266.6666667
30	7.9	65.83333333	260.0416667
35	7.9	65.83333333	260.0416667
40	7.9	65.83333333	260.0416667
45	7.9	65.83333333	260.0416667

50	7.8	65	253.5
55	7.8	65	253.5
60	7.8	65	253.5

Figure 2.3.3: Capacitor Discharge over time while MOSFET gate is closed.

According to the figure above, over an hour span, the capacitors didn't lose a significant amount of charge, and the rate of discharge seemed to decrease as time went on or as the actual amount of voltage in the capacitors decreased. If it was estimated that 7W of power would discharge in a second or so, then there was still an overflow of energy more than needed, and it was realized that there was hardly a need to worry about capacitors leaking charge without the MOSFET gate "opening" as long as the capacitors were charged to 8V or so.

Now, the entire cut-down mechanism was ready for action, and so in the weekend before the tethered launch, the system was taken for testing. The mechanism worked in all of this testing but failed in a quick test before the tethered launch. It cannot be fully certain, but it is likely that the connection from wire to nichrome was slightly too inconsistent (as this was before the start of the crimping process), and thus the circuit was rebuilt, the solder connections were super-tested, and the crimping process was diligent to prevent this from happening again.

Without the tethered launch as a guide, more extensive testing was needed in the pressure and temperature chamber to ensure the cut-down system worked up in the stratosphere. So, after many tests, the cut-down mechanism was inserted into the final code and was ready for launch day.

The mechanism did not work, but that is due to other circumstances beyond the scope of this section. In short, some of the pressure sensor calibrations didn't quite make the final code, and thus, the trusted and integral pressure readings were off and prevented the cut-down mechanism from firing at all.

After the payload was recovered, the capacitors were still charged over 7V, confirming that if the cut-down mechanism was triggered, the results would've been as expected: successful. However, this is only because of the extensive testing and care from the entire team to ensure that everything was ready for launch day.

2.4 Buoyancy Calculations

The buoyancy force acting on the balloon was calculated using Archimedes' principle, which states that the upward force on an object equals the weight of the fluid it displaces. Because helium is lighter than air, the balloon is able to float. The calculations below use this idea to find how strong the upward push is, and then subtract the weight of the balloon and everything attached to it to find the actual lift force.

Buoyancy Force:

Buoyancy force is found by using the following equation:

$$F_B = p_{net} * V * g$$

Symbol	Definition	Value
ρ_air	Density of air at sea level	1.225 kg/m³
ρ_He	Density of helium at sea level	0.164 kg/m³
ρ_net	ρ_air − ρ_He (effective buoyant density)	1.061 kg/m³
r	Balloon radius (3 ft ≈ 0.914 m, rounded to 1 m)	≈ 1.0 m
V	Balloon volume, modelled as a sphere: (4/3)πr³	≈ 4.19 m³
g	Gravitational acceleration	9.8 m/s²

Figure 2.4.1: Defining Variables

Step 1: Calculating the density

Since air is denser than helium, the balloon is able to float. To find how much "lift potential" the air can give us, we subtract the density of helium from the density of air. This gives us the net density, which shows how much heavier the surrounding air is compared to the helium inside the balloon. A bigger difference means more lift.

$$p_{net} = 1.225 - 0.164 = 1.061 kg/m^3$$

Step 2: Calculating the volume of the balloon

Because the balloon is roughly spherical, we used the formula for the volume of a sphere. The balloon's radius is 3 feet, which we approximated to 1 meter. Calculating with this radius in mind, the volume is about 4.19 cubic meters. This is roughly the amount of space the balloon takes up in the air around it.

$$V = 4/3 * \pi * 1^3 \approx 4.19 m^3$$

Step 3: Calculating the buoyancy force

Now we multiply the three values together: the net density, the volume, and gravitational acceleration ($g = 9.8 m/s^2$, which is the standard pull of gravity). The result tells us how much force the air is pushing the balloon upward.

$$F_B = 1.061 * 4.2 * 9.8 \approx 43.56N$$

Total Upward Force at Maximum Altitude:

The total upward force represents how much lift is available once the weight of our payload is accounted for. The net force tells us what is left over to pull the balloon upward.

$$F_{Net} = F_B - mg$$

Step 4: Calculating the weight of our payload

The total mass of the payload, which includes the balloon, the payload, and our circuits, was measured at 3.65lbs or 1.656kg when converted to metric units. Multiplying this by the gravitational acceleration gives us the downward gravitational force, which is the weight the balloon has to overcome.

$$Mg = 1.656 * 9.8 = 16.23N$$

Step 5: Calculating the net upward force

Finally, we subtract the weight of the payload from the buoyancy force. What remains is the net upward force, which is the lift.

$$F_{Net} = 43.56 - 16.23 \approx 27.33N$$

This result confirms that at maximum altitude the balloon experiences a net upward force of 27.33N, demonstrating sufficient lift capacity for our payload.

Sensors and Calibration

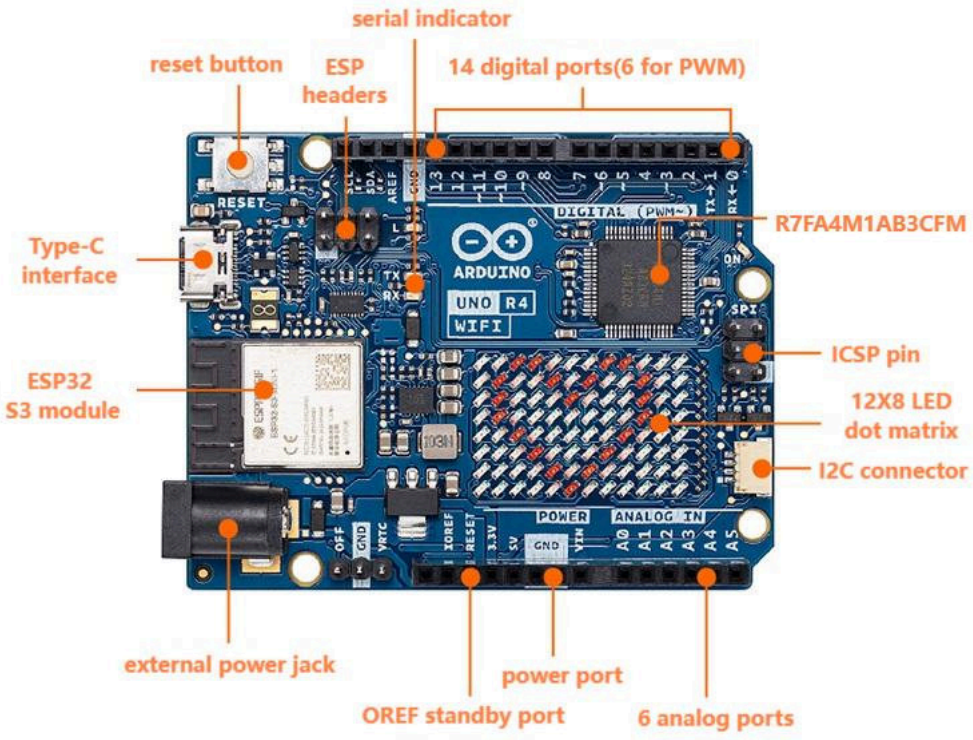


Figure 3.0.1: Diagram of Arduino Uno R4, used as the central microcomputer in the payload (Credit: keystudioWIKI)

3.1 Sensor Overview and Arduino Configuration

Arduino Uno:

The Arduino is a microcontroller platform that is designed to be compatible with sensors and control external hardware. At its core, the Arduino executes code written by the user to read inputs from sensors and produce outputs. Both input and output use either digital or analog. Digital signals come in either low (0V) or high voltage (5V) which are converted into binary, while analog signals range from 0V to 5V in a continuous manner and is converted into numbers

using an internal ADC. The versatility and programmability of the Arduino make it perfect for data collection systems such as the payload.

Vernier GPS-BTA Analog Pressure Sensor:

The Vernier GPS-BTA pressure sensor measures atmospheric pressure by using a piezoresistive element. The pressure-induced strain causes this element to change resistance, and this change is then translated into an analog voltage output that is read by the Arduino. Because of the predictable relationship between atmospheric pressure and altitude, this sensor can be used to effectively determine altitude throughout the flight of the balloon. The analog output also allows for continuous and high-resolution sampling.

10k Thermistors:

Thermistors are resistors that have variable resistance depending on the temperature. The 10k thermistors used in this experiment are negative temperature coefficient (NTC) devices, which means their resistance will decrease as the temperature rises, and vice versa. When implemented into a voltage divider, the thermistor produces voltage readings that can be translated by the Arduino's ADC to calculate the actual temperature. However, this relationship between resistance and temperature is not linear, and is instead modeled by the Steinhart-Hart equation.

Adafruit Ultimate GPS Logger Board:

The Adafruit Ultimate GPS Logger is a satellite-based positioning system that determines location through signals from GPS satellites. It is able to calculate latitude, longitude, altitude, and time by triangulating the time delay (how long the wave takes to reach it) from different

orbiting satellites. This module also has a built-in data-logging feature, which allows for long-term storage onto a microSD card.

Teensy:

The Teensy is a compact microcontroller board that serves a similar purpose to the Arduino, but acts as a backup system in case the main Arduino board malfunctions.

MS5803 Pressure and Temperature Sensor:

The MS5803 is a high-resolution barometric pressure sensor that also measures temperature. It uses a microelectromechanical system (MEMS) to sense pressure. This component deforms under the applied pressure, and that deformation is converted into a digital signal through an onboard ADC. This component was chosen for its high precision, making it suitable for determining altitude in this balloon experiment.

BME680 Temperature, Humidity, and Pressure Sensor:

The BME680 is an environmental sensor that is able to measure all three of temperature, relative humidity, and barometric pressure. It utilizes a temperature sensor with a bandgap circuit, a humidity sensor that senses changes in dielectric properties due to moisture, and a MEMS-based pressure sensor. The versatility of this sensor makes it very valuable for obtaining very different types of data simultaneously.

3.2 Sensor Calibration

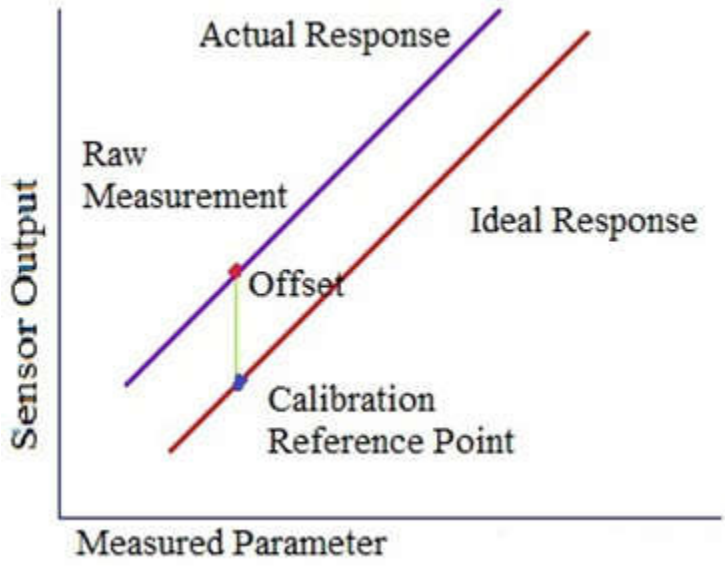


Figure 3.2.1: Graph representation of generic sensor calibration (Credit: Semiconductorforyou.com)

Temperature

Thermoresisrtor	-60ºC	21ºC
Inside Measured	492939Ω	11900Ω
Inside Predicted	1957000Ω	11900Ω
Outside Measured	121001Ω	11500Ω
Outside Predicted	1891972Ω	11900Ω

Figure 3.2.2: Table of predicted and measured resistances used for thermistor calibration

Equation for two-point linear calibration: $T_{actual} = m \times T_{reported} + b$

Where: m = slope = $(T_actual_2 - T_actual_1) / (T_reported_2 - T_reported_1)$ b = offset = $T_actual_1 - m \times T_reported_1$

Outside Thermistor:

Point 1: Reports 70°F, actual 70°F Point 2: Reports -8°F, actual -76°F (-60°C)

$m = (T_actual_2 - T_actual_1) / (T_reported_2 - T_reported_1)$

m = (-76 - 70) / (-8 - 70) m = -146 / -78 m = 1.872

$b = T_actual_1 - m \times T_reported_1$

b = 70 - 1.872 × 70 b = 70 - 131.03 b = -61.03

T_actual = 1.872 × T_reported - 61.03

Inside Thermistor:

Point 1: Reports 70°F, actual 70°F Point 2: Reports -45°F, actual -76°F (-60°C)

$m = (T_actual_2 - T_actual_1) / (T_reported_2 - T_reported_1)$

m = (-76 - 70) / (-45 - 70) m = -146 / -115 m = 1.270

$b = T_actual_1 - m \times T_reported_1$

b = 70 - 1.270 × 70

b = 70 - 88.87 b = -18.87

T_actual = 1.270 × T_reported - 18.87

To calibrate the temperature thermoresisros, 2 points were plotted at different temperatures, and the Steinhart equation was used. First, the thermoresistors were placed in a freezer at -60ºC for 5 minutes. A thermometer was used to verify the temperature, and then the raw value was recorded. Next, the thermistor was taken out of the freezer and exposed to ambient temperature for 5 minutes. The ambient temperature was then recorded, and the raw value was logged. These values were plugged into the Steinhart Equation with B = 3950.

The code was written in such a way that by changing variables such as slope or intercept value, the sensor could be calibrated. This was true for both the thermoresistors and the analog pressure sensor. After plugging in the values determined by the Steinhart Equation, the sensors were tested again and resulted in values less than 1% off from the actual measured values.

Pressure

Pressure Sensor	@ sea level (0ft)	@ 100,000ft
Predicted (ISA)	1013.25 mbar	10.31 mbar
	2.577v	0.026v
Measured (Analog)	937.6 mbar	194.8 mbar
	2.384v	0.495v

Figure 3.2.3: Table of predicted and measured values of the analog pressure sensor used for calibration

Equation for two-point linear calibration: $P_{actual} = m \times P_{reported} + b$

Point 1: Ground level, sensor reads 937.6 mbar, actual 1013.25 mbar Point 2: GPS peak 100,126 ft, sensor reads 194.8 mbar, ISA actual 10.42 mbar

$m = (P_{actual_2} - P_{actual_1}) / (P_{reported_2} - P_{reported_1})$

$m = (10.42 - 1013.25) / (194.8 - 937.6)$ $m = -1002.83 / -742.8$

$m = 1.350$

$b = P_{actual_1} - m \times P_{reported_1}$

$b = 1013.25 - 1.350 \times 937.6$

$b = 1013.25 - 1265.82$ $b = -252.57$

$P_{actual} = 1.350 \times P_{reported} - 252.57$

To determine the pressure calibration metrics, a similar method was employed as the temperature calibration procedure. Because the vernier GPS-BTA pressure sensor used is designed to scale linearly, a 2-point calibration system was used. Originally, the pressure sensor was placed in an ambient atmosphere, and the raw value was recorded. Then, the pressure sensor was placed into a vacuum chamber where the pressure was lowered to 0 mbar, and then raised to 10mbar, where the voltage was recorded at both intervals. Then the record voltages were imputed into the above equation, and the offset + slope were calculated. These values were put into the code and verified with more tests.

3.3 Sensor Related Errors

Data Anomalies

Interestingly, whenever the Arduino was reset, the Verneir GPS-BTA would report garbage data until it was unplugged and replugged in. The sensor would act as if it were not connected to the Arduino until it was replugged back in. The team was unable to determine the cause of this error, but this did not present an issue on launch day, as the team knew to unplug and replug the sensor after the Arduino had been powered on.

During the flight, the GPS chip also experienced a significant area of signal loss. This signal loss occurred during the rapid descent, where the payload fell at speeds over 60m/s for over 13 minutes. At 58000 ft, the chip lost signal and continued reporting that same value for multiple minutes. A signal was received again at 5826 ft, and the chip failed to report accurate altitude data all the way up until 30 ft when it collided with the ground at over 40 miles per hour, and the SD card was ejected from the Arduino. The cause of the GPS signal loss is still unknown, but the rapid fall and tumbling is the prominent theory. Below is the graph of GPS altitude and the number of satellites the GPS was connected to at different times throughout the flight.

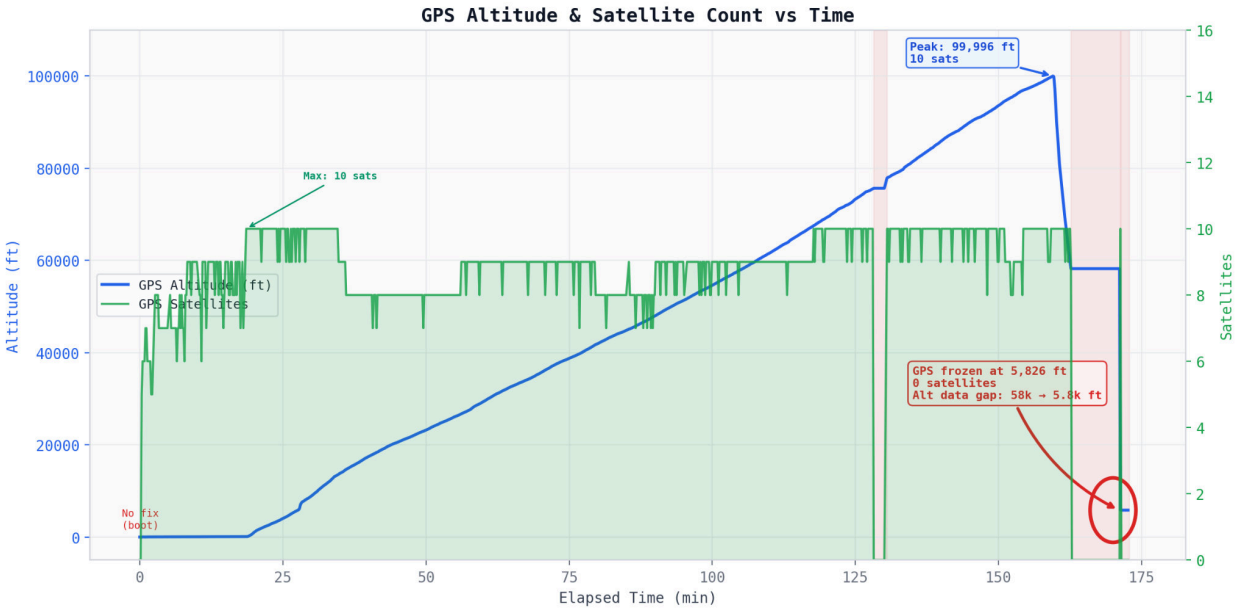


Figure 3.3.1: Graph of GPS altitude and number of GPS satellites vs time to highlight signal loss during descent

Before the day of launch, many sensors failed to report reliable data during testing. Most notably, the team encountered errors with the MS5803 pressure sensor. Similar to other groups, it was observed that the sensor would perform normally for the first few minutes/hours, but after some time, it would stop reporting data altogether. Upon further investigation, it was observed that after use with the arduino the sensors would internally short across the Vcc and Gnd terminals, causing the Arduino to be unable to report data at all. Shortly after this discovery, the team pivoted to start using analog pressure sensors on the Arduino.

Calibration Errors

The major issue of the launch resided in the calibration of the analog sensors. On the day of the launch, the code that was uploaded to the balloon did not contain the updated calibration values, which led to skewed data. By reverse engineering the raw data values using the coded calibration equations, the team was able to recover the actual data. However, this error led to many corollary issues, such as the cutdown not firing and the heating pad not triggering when it was supposed to. When the pressure was 10mbar, the sensor would read .5 volts, which the Arduino interpreted as ~40000ft, well below the actual value. The same issues with calibration resulted in a higher interpreted temperature within the payload. Because the heating system was triggered based on the interior thermoresistor, the heating pad did not trigger until the descent, where the payload experienced rapid descent and air resistance.

Below are table of the values of pressure, inside temperature, and outside temperature with both the values before calibration (what the Arduino read during launch) and after calibration (actual values)

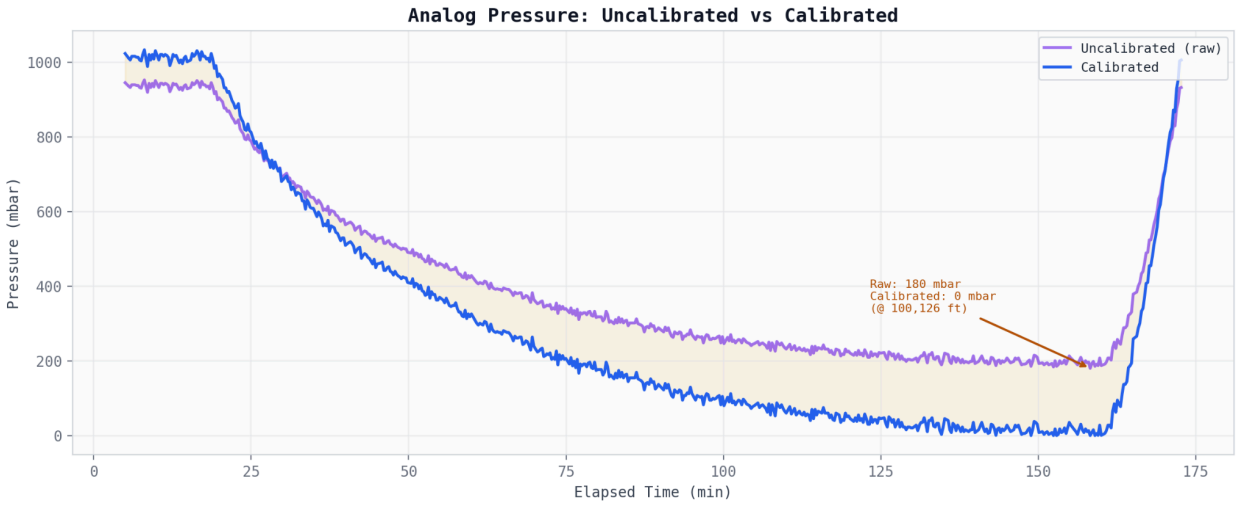


Figure 3.3.1: Graph of Pressure vs time for uncalibrated and calibrated values

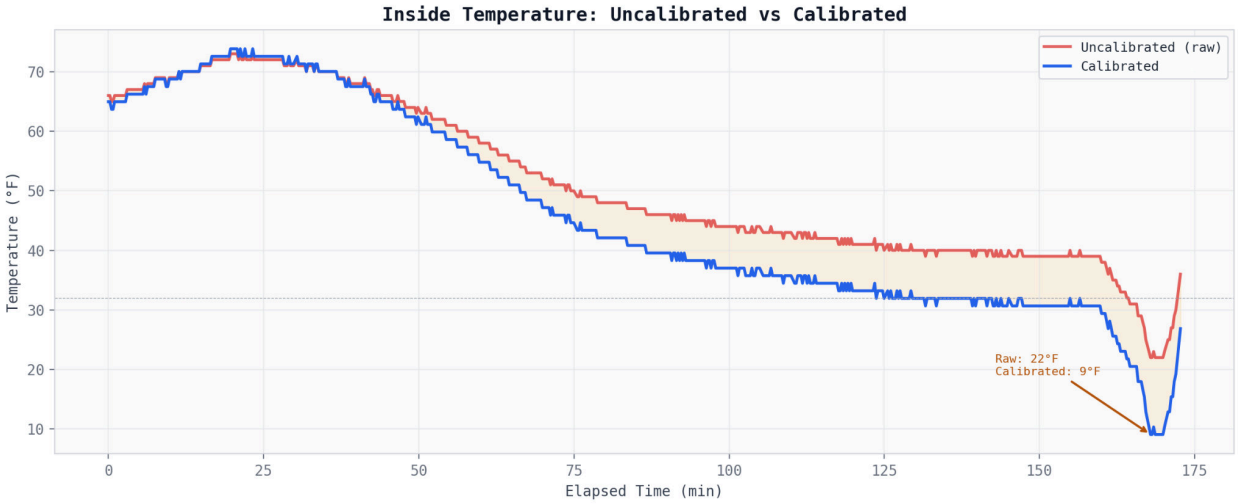


Figure 3.3.2: Graph of Inside temp vs time for uncalibrated and calibrated values

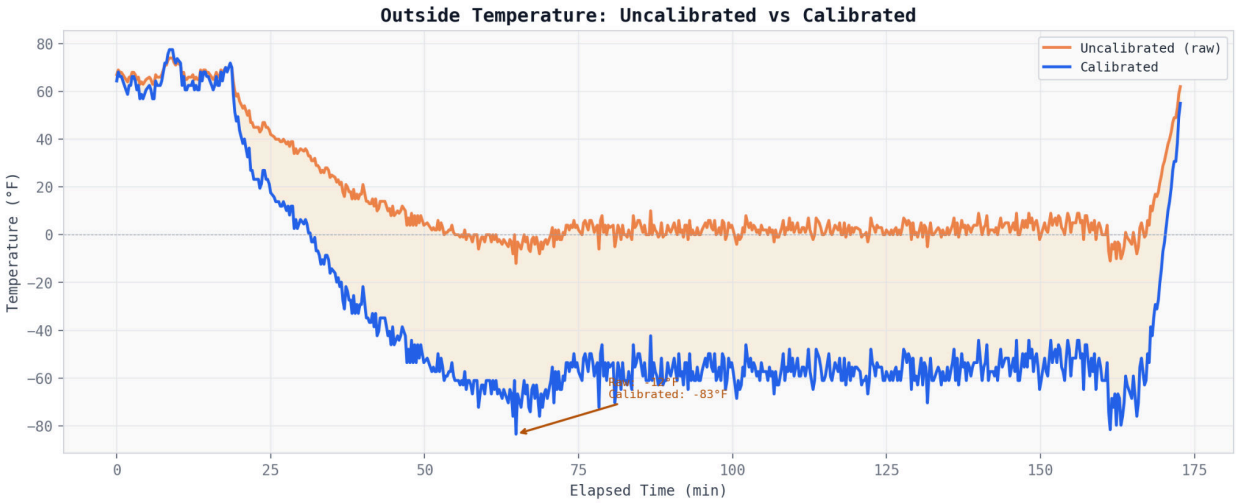


Figure 3.3.3: Graph of Outside temp vs time for uncalibrated and calibrated values

Radio and Tracking Systems

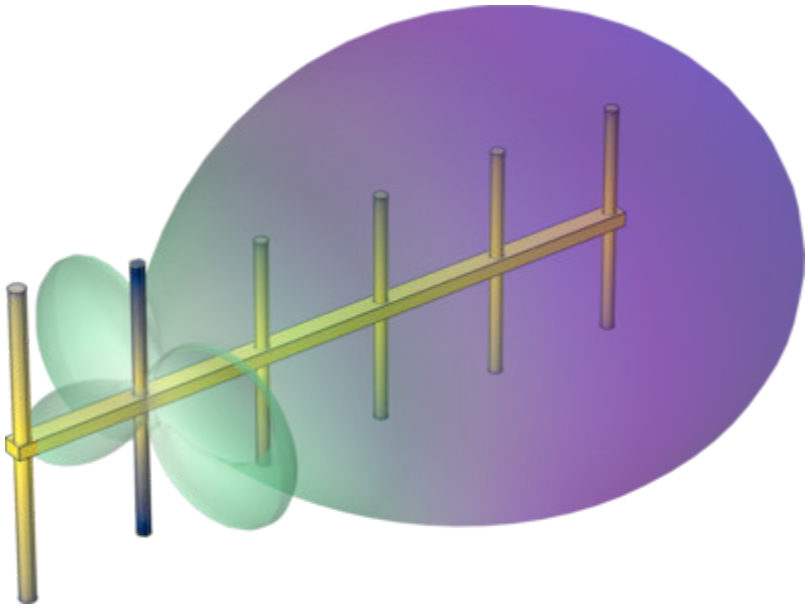


Figure 4.0.1: Diagram of Yagi antenna with radio fields (Credit: Comsol)

4.1 Yagi Antennas and Foxhunt System

A Yagi antenna is a type of antenna that optimizes the gain or range of a signal in exchange for an omnidirectional function. It consists of 3 metal poles (we used tape measure metal), which make up different components: the director, driven, and reflector. We use the yagi antenna in this setting as a fox hunt device. In our payload, we put a beacon which emits radio waves at 146.545MHz, and using the directionality of our yagi antenna, we can determine which direction our payload is in and find it by going that way. Additionally, the yagi antenna's extra range helps with this function, as having just a normal omnidirectional antenna wouldn't work as well due to the extreme distances a payload could travel.

The driven element is the only part of the Yagi antenna that is actually connected to the ham radio. This is the main “antenna” which picks up the frequency that our beacon emits. Without the other elements, this could act as an omnidirectional antenna optimized for our beacon frequency. Basically, the beacon sends out an electromagnetic signal which pushes charges back and forth in the metal antenna; this creates alternating current in the metal, which gets delivered to our ham radio through the coax cable. The metal piece resonates best at half the wavelength of the beacon, as the radio waves would create a standing wave of current. I also cut the driven element just 5% shorter than exactly half the wavelength because I found that the antenna behaves electrically a little longer than it actually is due to edge effects around the ends of the antenna. To actually make the driven element, I used two separate pieces of tape measure, which I connected by soldering together with a coax cable, the inside of the cable to one piece and the outside to the other. I then added a copper bridge to connect the

The reflector element goes behind the driver and prevents signals that come from behind from being heard. It essentially does this by sending a small reradiated electromagnetic wave that adds to the signal in front and subtracts from the signal in the back through constructive and destructive interference. The reflector can accomplish this because it’s slightly longer than the driven element, and when the metal rod is longer than its resonant length, it reradiates a phase-shifted version of the electromagnetic wave given to it. This then can cancel out the original wave if the antenna is pointed backward, and amplify it if the antenna is pointed forward. I tried different reflector lengths like 1% and 20% larger than the driven, but I ultimately settled on around 5%; both the 1% and 20% weren’t canceling waves well enough because the 1% didn’t send the wave out phase enough, and the 20% was too far off resonance to

induce enough reradiated waves to be useful. The distance between the driven and reflector is 0.2 wavelengths.

The director element works in a similar way to the reflector by reradiating given waves with a phase shift. Because the director is shorter than the driven instead of longer like the reflector, its reradiated waves have a phase lead. By the time the reradiated signal gets to the driven, the signal can constructively interfere with the original signal to amplify the signal in the forward direction.

Then, detailed line of sight tests and Obstruction tests (i.e. how far is the range). While I (Lucas) was absent, Tyler and Ethan took some tests on my behalf. They first got at the end of a straight street, then Tyler began to drive with the radio as Ethan held the antenna. Once they lost signal, Tyler stopped, took note of the location, and returned. This process was repeated three times. After these line of sight tests, we found that our range was 0.26 miles. For obstruction tests, Tyler turned down a street with numerous houses and trees between the radio and the antenna in the same process. After three trials, we found the obstruction range to be 0.17 miles.

4.2 Pico APRS and GPS

Radio signals are a form of light wave. There are two main features to a radio signal, which are the frequency and modulation (in this case, FM). The frequency of a radio signal can separate different stations or signals from each other, like lanes on a highway, so that different signals can be identified and read at the same time. In a certain frequency, information packets still need to be transmitted through the signal, which is where modulation comes into play. There are two types of modulation: amplitude and frequency, but we only use frequency modulation. In

frequency modulation, the given signal is either slowed down or sped up (increased or decreased in frequency) to signal different information. A very rudimentary system would be a faster signal for a 0 or a slower signal for a 1, essentially creating binary code through modulations.

Specifically, the PicoAPRS v3 is tuned to broadcast and receive signals at 144.390MHz. Then, very small modulations are made to the carrier wave, meaning that the signal goes slower to faster over and over at a certain modulation frequency. The FM modulations the PicoAPRS uses are 1200Hz for a 0, and 2200Hz for a 1. This can then generate binary code, which can be translated into numbers or data.

GPS is short for global positioning system, and it's a centralized system to determine where things are on the Earth. GPS is measured through coordinates, which are determined by latitude and longitude. Because the world is spherical, degrees are used as units. Latitude (north or south of the equator) is measured in level circles, so anything at the same level circle has the same latitude. On the other hand, longitude measurements are circles which go from the North Pole to the South Pole, rotated at a certain angle. When we have latitude and longitude, we know exactly where something is based on the intersection of the latitude circle and the longitude circle.

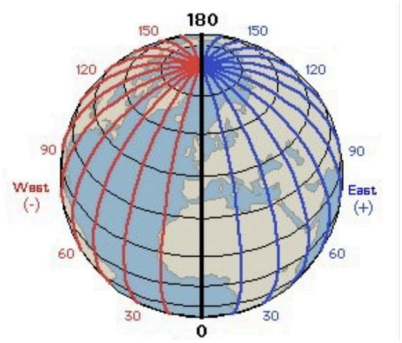


Figure 4.2.1: diagram of Earth with latitude and longitude (Credit: schoolworkhelper.com)

The way GPS is determined is a little different. Take a certain position on the Earth; to find how far away from a satellite it is, the satellite will broadcast a signal to a receiver (in our case, the PicoAPRS), giving its precise location in orbit and time based on an atomic clock. Then, based on the time displacement between the moment the satellite sent the signal and the moment the Pico received the signal, we can calculate how far the satellite was from our Pico the moment it sent the signal. Using that information, we know that our Pico is somewhere on the surface of a sphere centered around the satellite location with a radius of the distance between the satellite and Pico. Then, with multiple satellites in different locations, we get a multitude of different spheres; at the intersection of these is our absolute GPS location. Please refer to the diagram below for a simplified two-dimensional model of GPS locationing. GPS works best with at least 4 satellites connected.

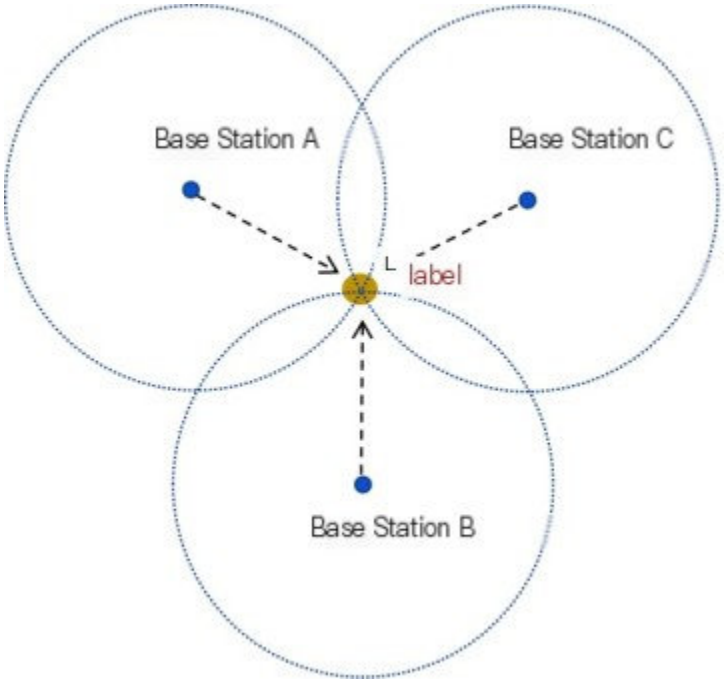


Figure 4.2.2: 2-dimensional diagram of satellite triangulation model (Credit: NiceRF)

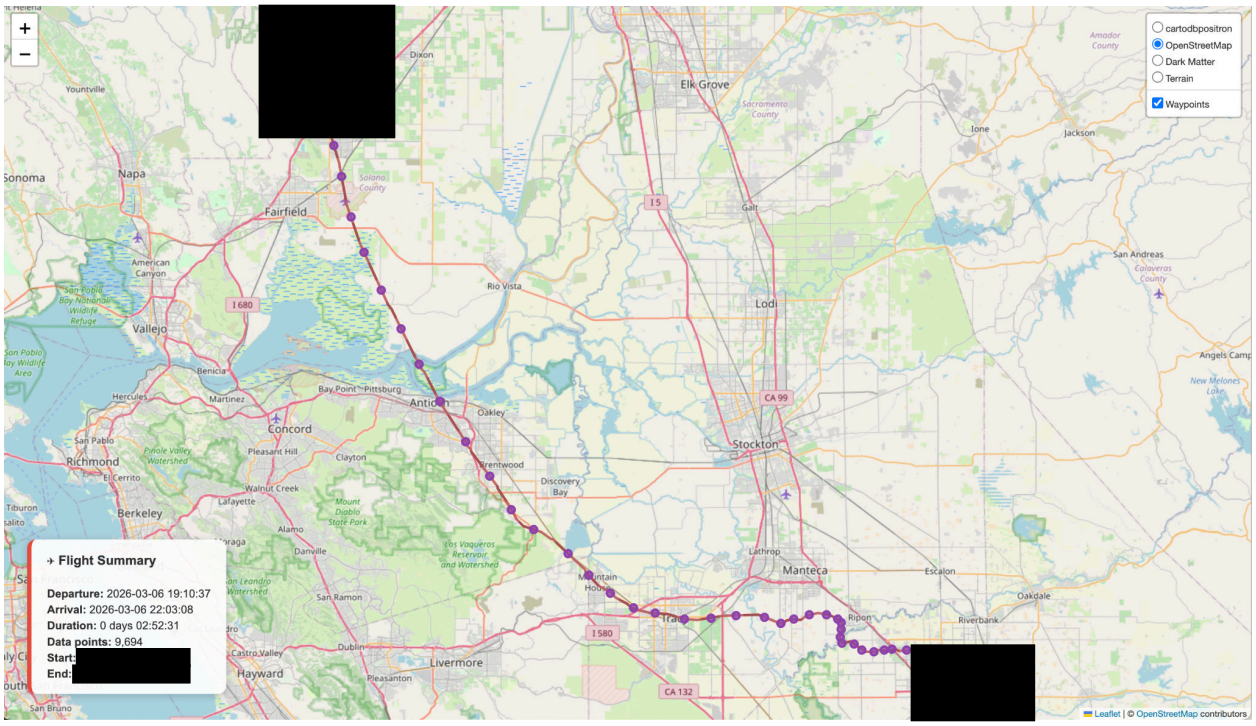


Figure 4.2.3: Map of Payload tracked by onboard GPS

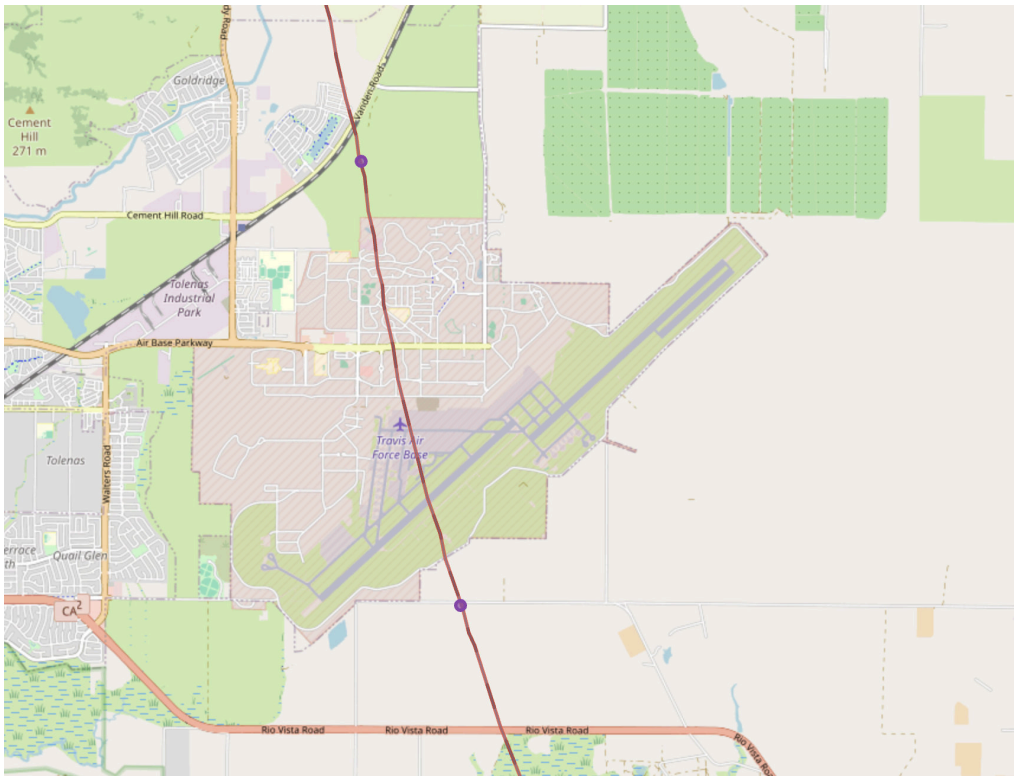


Figure 4.2.4: GPS position as payload flies over Travis AFB

4.3 Beam-Down System

The payload was outfitted with a radio system that would allow the wireless transmission of data from altitude down to a HAM radio via digital frequencies. The system used a 100mW analog transmitter that would send frequencies at 151.300 MHz every 5 minutes. The concepts of how the radio worked were very similar to the foxhunt (see section 4.1), yet it transmitted at significantly lower frequencies, which allowed for a much longer range. In testing, the radio was able to transmit down from 1000 ft, proving that data could be transmitted reliably.

To transmit the data, RTTY digital packets were used. Originally, the team had used Morse code to transmit the data. However, to transmit the data reliably, it would take more than 2 minutes per transmission. Furthermore, during testing, the team noticed a reated clicking noise interfering with the transmission. This clicking was traced back to the switching of pinouts, such as the indicator LEDs or the data logging pins. The team discovered that during the period of transmission, in order to avoid background noise from the switching of pinouts, the Arduino could not perform functions such as data logging. The Arduino was programmed to go "silent" while transmitting so the signal would come through clean. However, if Morse was used, the payload would lose upwards of 30% of its data.

To solve this, RTTY digital transmission packets were used. RTTY stands for Radio Teletype, and it works by switching between two audio tones to represent binary data. One tone (800 Hz) represents a "mark" or binary 1, and a second tone (970 Hz) represents a "space" or binary 0. The system was configured to transmit at 45 baud with 8 data bits and 2 stop bits. Baud refers to the number of signal changes per second, so at 45 baud, the system was sending 45 bits per second. 8 bits means each character is encoded as 8 binary digits, which is enough to

represent all standard ASCII characters (letters, numbers, symbols). This allowed a full data packet containing GPS coordinates, altitude, pressure, temperature, and humidity to be transmitted in approximately 15 seconds rather than the 2 minutes Morse code required. On the receiving end, the signal was picked up by a HAM radio and decoded using a program called Fldigi, which is an open-source digital modem application. Fldigi was able to listen to the audio coming out of the radio, identify the two RTTY tones, convert them back into binary, and display the decoded text on screen in real time. This meant the team could theoretically monitor live telemetry from the payload throughout the entire flight.

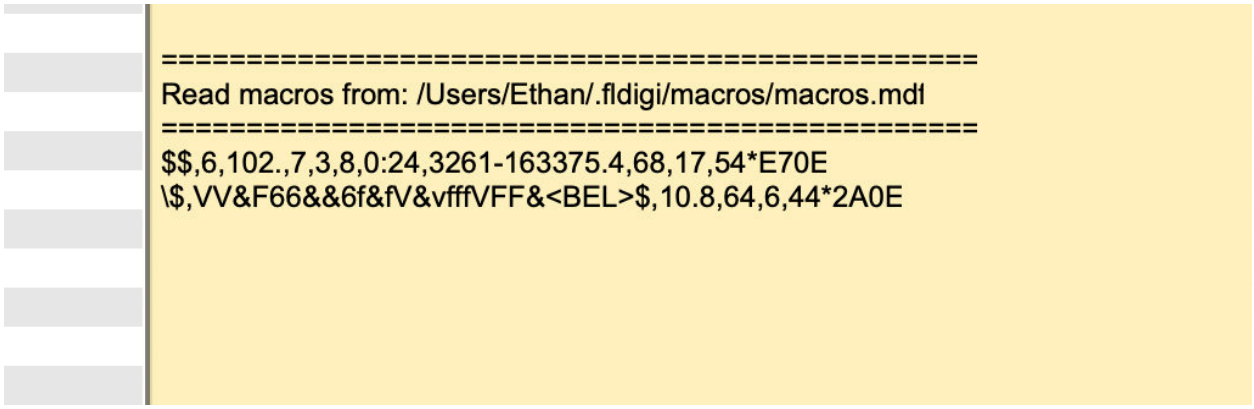


Figure 4.3.1: Screenshot of Fldigi with raw transmitted beam-down data

Despite successful tests, the beam-down system only transmitted data for the first 25 minutes of flight. The decoded messages were all partial, and very few of them contained more than 1 or 2 points of data. In Figure 4.3.1, one can see the actual deciphered messages from the transmission system. This data, when decrypted, showed that the payload was reading an inside temperature of around 68°F, humidity around 54%, and what appeared to be an altitude of roughly 3,261 meters. These values lined up closely with what the SD card logged at the same point in the flight, confirming that the radio system was in fact transmitting real sensor data. The

rest of the characters were undecodable due to signal degradation at range, which is expected behavior for a 100mW transmitter pushing RTTY through the atmosphere. Despite the data being corrupted over a wide range, a signal was still readable even at over 80 miles away, showcasing the power of the system.

Decoded RTTY Breakdown (successful packet):

Below is the breakdown of how the data from Fldigi was interpreted.

Packet format: $$,txCount,HH:MM:SS,lat,lon,altM,press.p,thermIn,thermOut,hum*CRC16

First line received: $$,6,102.,7,3,8,0:24,3261-163375.4,68,17,54*E70E

Field	Received	Interpretation
$$	$$	Packet start marker
txCount	6	6th transmission (~30 min in)
time	102.,7,3,8,0:24	Garbled, likely 19:40:24
altitude	3261	~3261 m (garbled)
pressure		Corrupted
inside temp	68	68°F
outside temp	17	17°F (corrupted)
humidity	54	54%

CRC	*E70E	Checksum

4.4 Radio Related Errors

During the real-world launch, there were multiple errors with the radio setup. Firstly and most importantly, the picoAPRS system wasn't working properly, and the team wasn't getting pings. Originally during the tethered launch, the Pico wasn't working properly either, but due to it immediately running out of battery when taken out of the payload, they assumed it was due to either a charging issue or battery contact issue, which they fixed by using a new, fully charged battery for the real launch and zip-tying together the back of the Pico. It was also working well the day before, as the team was getting regular pings around 10-15 minute intervals (not the ideal 60s they were looking for, but still good enough for the launch), and during drives home most days the Pico would be put in the car, where it would work relatively well.

During the launch day, they were having slight issues with the Pico as they didn't get a ping on [redacted] until 11am, and two consecutive (~30 interval) pings at 11:20 am. They launched around 11:30 am, and no signal was received after that. They expected to get a ping around 10-20 minutes after launch given previous circumstances, but suspect that due to flying directly over Travis Air Force Base, the GPS systems were jammed and the pings they were supposed to get never came. They definitely could have tested the PicoAPRS more rigorously to make sure it was working, but even if the Pico had been working properly like other groups, they still would have lost signal due to the military base interfering with communications.

Although there was never an opportunity to test the Yagi antenna foxhunt in the real launch (as the payload was found from an Airtag hit), there is reasonable confidence that the team would have been able to find it with foxhunting had they gotten a Pico signal. The beacon started pinging in the bus right after they got off at Modesto, around 15 miles north of Turlock, where the payload landed, and with around 5-10 practice real-world foxhunts completed, the team would have been able to recover the payload had the Airtag not hit.

4.5 Predicted vs Actual GPS

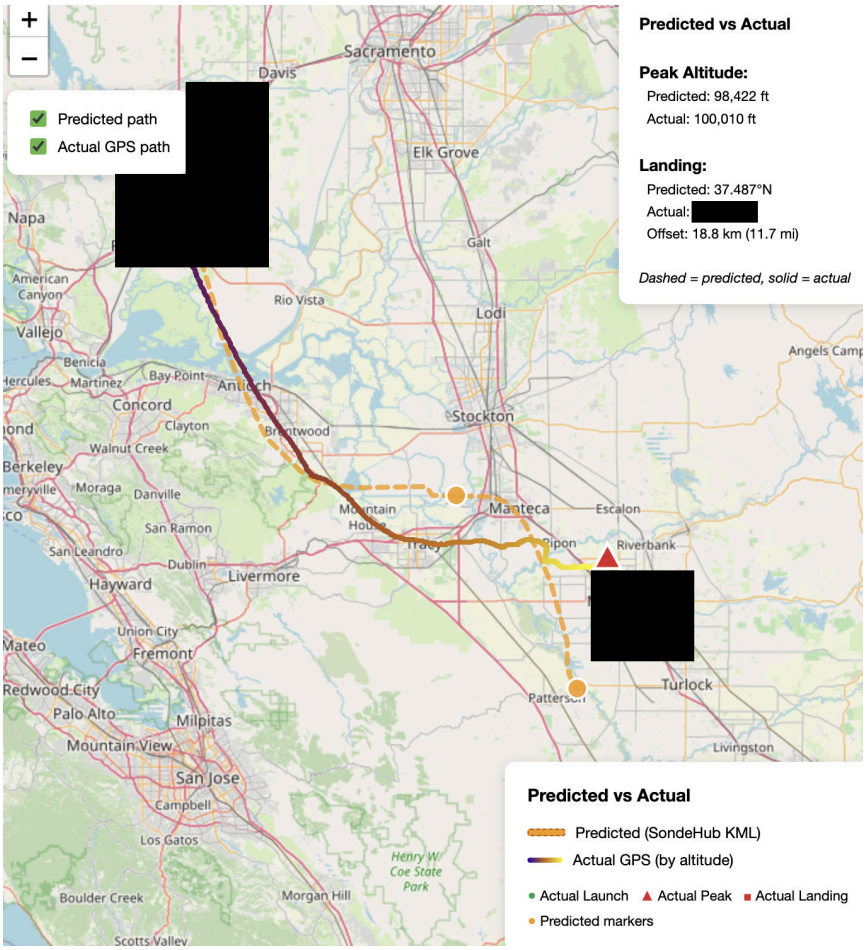


Figure 4.4.1: Predicted path vs actual path record by GPS for payload

The SondeHub prediction had the balloon landing near Patterson, about 11.7 miles south of where it actually came down near Modesto. The predicted peak altitude was 98,422 ft, while the actual GPS peak came in at 100,126 ft, which is only about 1.6% off. The balloon followed a similar general path heading SSE from the launch site, but the actual path drifted a bit more to the east than the prediction expected, likely due to slight differences between the forecasted upper-level winds and what was actually happening at altitude that day. Overall, the prediction was accurate enough to confirm that the launch site and day were safe choices, and the payload landed well within the recoverable zone the team had planned for.

Data Analysis and Results



Figure 5.0.1: Summary of data collected (Credit: Claude AI)

5.1 Pressure vs Altitude

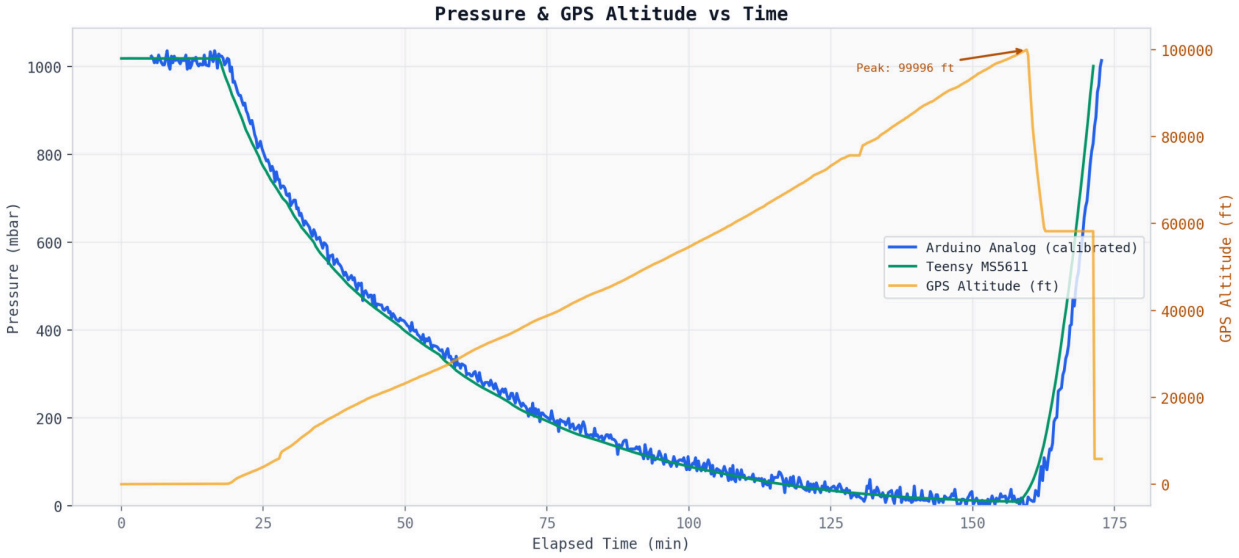


Figure 5.1.1: Pressure vs. Altitude during Ascent and Descent

This graph illustrates the relationship between the pressure in the atmosphere and the altitude, as measured during the entire duration of the payload's flight. As expected, the data shows a strong inverse relationship, where the increase in altitude leads to a decrease in pressure in the atmosphere. This phenomenon is clearly seen during the ascent portion of the flight, both the Arduino analog sensor and the Teensy MS5611 sensor data show a smooth exponential decay in pressure. Although there is a small amount of noise in the Arduino analog sensor data, both of the pressure sensors are consistent in their reports.

As the ascent continues, the curve gradually becomes flatter as the height increases. This is reasonable since the rate at which the pressure is decreasing is slower as the height increases. The majority of the mass in the atmosphere is close to the surface of the earth. Hence, most of the change in pressure occurs close to the surface.

The second part of the graph shows the descent process. At this time, after reaching the maximum height (around 100,000ft), the rate at which the pressure is increasing is much sharper

compared to the ascent process. This is due to the fact that the parachute of our payload never went off, leading to the payload to descend in a rapid state. This is confirmed by the altitude data since the height is decreasing sharply in a short time frame. As the payload descends further and enters the denser parts of the atmosphere, the rate at which pressure is increasing is also sharper.

Derivation of the Exponential Pressure-Altitude Model:

The relationship between pressure and altitude may be derived from physical principles. The pressure at a particular height in the atmosphere is due to the weight of the material above that point in the atmosphere. Therefore, as height increases, the amount of material above that point decreases, leading for pressure to decrease.

Mathematically, this may be expressed by the hydrostatic equilibrium equation:

$$dP/dh = -pg$$

Where P is pressure, h is altitude, p is air density, and g is gravitational acceleration.

However, in order to obtain this in terms of pressure, it is necessary to use the ideal gas law:

$$p = P/RT$$

Where R is the specific gas constant and T is temperature. Substituting this into the hydrostatic equation gives:

$$dP/dh = -Pg/RT$$

Rearranging terms:

$$1/P\ dP = -g/RT\ dh$$

Integrating both sides:

$$\int 1/P\ dP = \int -g/RT\ dh$$

$$\ln P = (-g/RT)h + C$$

Exponentiating both sides:

$$P = e^{(-g/RT)h + C}$$

$$P = Ce^{(-g/RT)h}$$

Letting $C = P_0$, the pressure at sea level, the equation becomes:

$$P = P_0e^{-h/H}$$

Where $H = RT/g$ is referred to as the scale height of the atmosphere, which is approximately 8.4km for the Earth.

This last equation indicated that the pressure in the atmosphere drops exponentially as the height increases. Although this theory assumes constant temperature and gravity, it is a good estimate of the behavior of the real atmosphere.

Comparison with the NASA Empirical Model:

Although the above exponential model is a simple and effective way to calculate the pressure at any given height, NASA uses an even more precise formula called the empirical model. This is because the temperature in the atmosphere is not constant. Temperature varies in the layers of the atmosphere. As the pressure and temperature have a direct relationship, the varying temperature affects the pressure curve. Despite this added complexity, both models agree that the trend in pressure is exponentially decreasing as altitude increases. The reason why pressure behaves this way in these two models is rooted in physics, where there is a lower number of gas molecules present at higher altitudes compared to sea level. There is less kinetic energy, meaning there is less collision between these molecules, which is what creates pressure. Therefore, as altitude is increased, pressure is reduced.

In summary, both models support the overall trend seen in the data, which is that the pressure exponentially decreases as the height increases. The data collected for the pressure

during the flight is consistent with the predicted behavior, especially during the ascent phase. The consistency between the theoretical models and the experimental data supports the validity of our data.

5.2 Humidity vs. Altitude

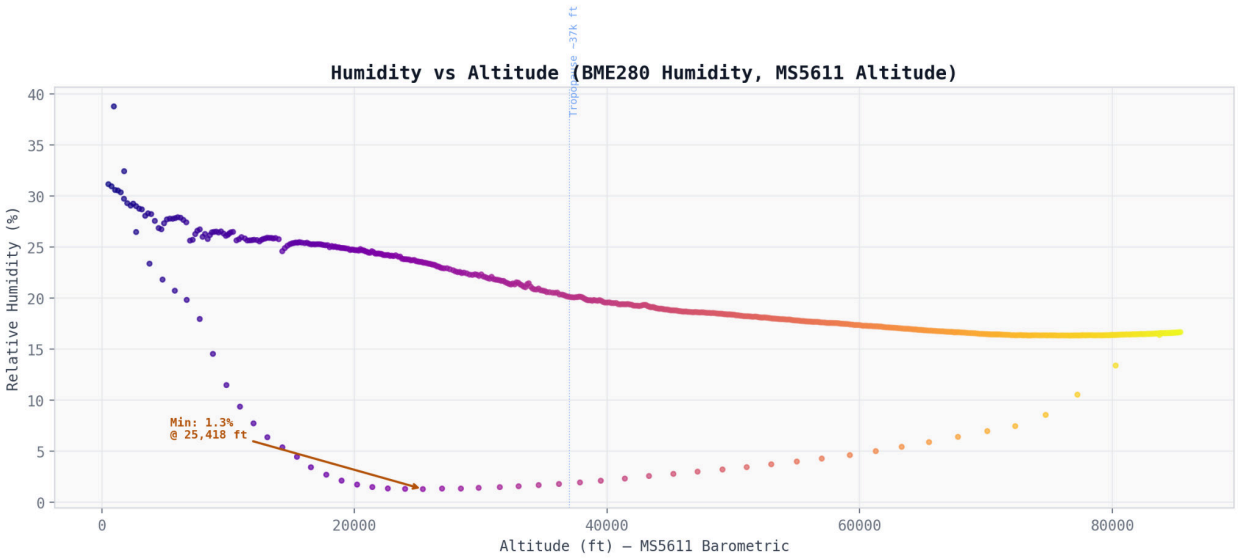


Figure 5.2.1: Humidity vs. Barometric Altitude during Ascent and Descent

This graph represents the change in humidity as a function of barometric altitude in the Earth's atmosphere. This graph depicts this relationship between humidity and altitude as related, because the graph seems to follow a similar curve during the relatively consistent ascent pattern of the weather balloon. However, the dotted line below is far more confounding, because it seems to bottom out somewhere pretty random. This graph depicts exactly what's expected: humidity decreases as altitude increases. Because humidity is related to temperature, and any mountain-lover knows the difference in mountain cold compared to that closer to sea level, these results are perfectly expected (at least on the ascent).

The descent is a little more confusing. Unlike prior ASR weather balloons, the parachute on this payload never deployed, and thus the entire apparatus free-fell from space. This caused extreme cold, and thus very low humidity readings. After reaching terminal velocity, it seems that the payload eventually encountered warmer and more humid environments as it reached closer to the surface.

It's actually quite insightful just how well this graph lines up with the expected reality, as the first 20,000 or so feet appear to be inconsistent, which is likely due to clouds and other factors that only exist in the nearest proximity to Earth's surface. Around the Tropopause (thin blue vertical line on the graph), the data gets a little inconsistent and flattens, which makes sense considering this layer acts as a literal "pause" in atmospheric layers. In this pause, the air temperature doesn't decrease with altitude, unlike in each of the separated layers. This portion of the atmosphere, and many others revealed by this graph, really brings an essential gravity to the scale of this project. As always, there is general noise in the data due to external factors (though the general consistency of this graph is because this data is from the internal humidity sensor), but otherwise, this data is indicative of expected trends and has proved successful in this quest for a better understanding of the Earth's atmosphere.

5.3 Temperature

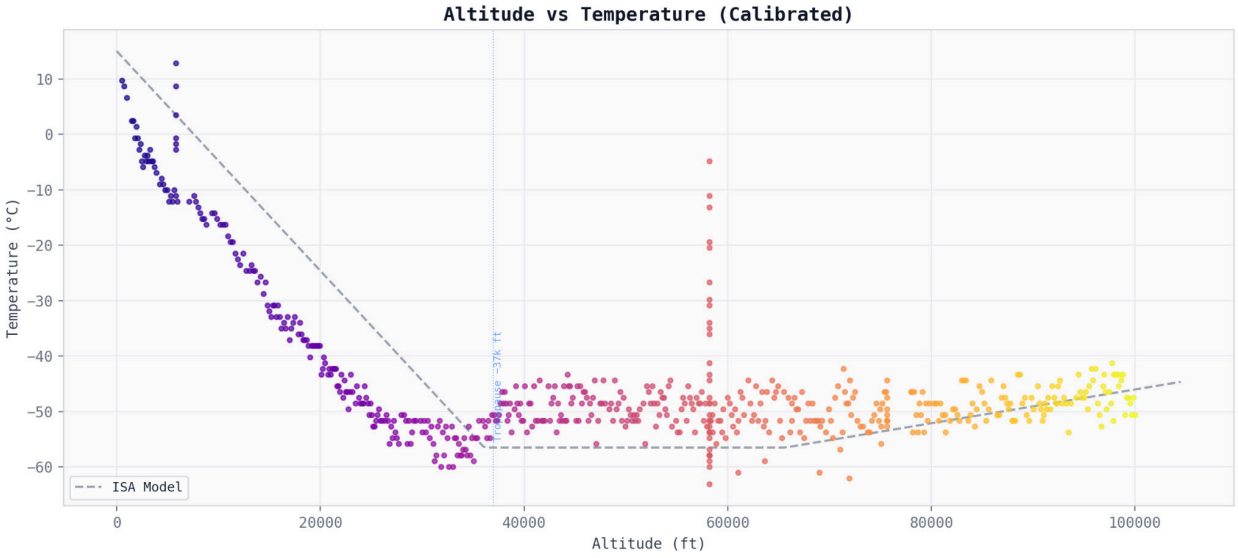


Figure 5.3.1: Temperature vs. Altitude during Ascent and Descent

This graph shows the relationship between the altitude and the calibrated temperature readings recorded during the entire flight, with the International Standard Atmosphere (ISA) model reference. The data points on the graph are colored from purple to yellow, visually indicating the different stages of the launch and the highest altitude reached, making it easy to distinguish the different stages of the flight.

As the payload rises through the Troposphere, the temperature drops from a minimum of 10°C at ground level down to a low of -60°C at about 30,000 to 35,000 feet. This is what is expected in this zone because, in the Troposphere, the temperature drops steadily with height. The data being recorded is in line with what is being modeled by the ISA model in this section, indicating that it is indeed reading accurately and as it should be.

At about 37,000 ft, the payload crossed into the Tropopause. The Tropopause layer is the point where the temperature level starts to stabilize after the consistent drop in the Troposphere. The ISA model shows that the temperature level stabilizes at about -56.6°C in this layer,

remaining constant after this point. Our data shows the same stabilization of the level after this point.

Above 37,000 ft, something that is not represented in the ISA model happens. While the ISA model shows a flat line, indicating constant temperature, our data shows a warming trend as the altitude continues to increase, with temperature rising back up towards -45°C as the altitude reaches its maximum height, approximately 100,000 ft. This makes sense because in the Stratosphere there is an increase in temperature due to the presence of UV radiation being absorbed by the ozone layer, which is not represented by the ISA model.

At around 60,000 ft of elevation, the graph indicates an increase in the scatter of the data points. This may be a result of the increase in the variation of the environment. At a higher altitude, there may have been an increase in the rate of change in temperature, a reduction in the density of the atmosphere, or an increase in the amount of solar radiation, which can introduce additional noise into our measurements.

Overall, this graph shows that the temperature sensor worked well throughout the flight. Comparing our data with the ISA model graph, it is consistent with what is known about the atmosphere and supports the validity of the temperature data received during our experiment.

5.4 Ascent/Descent Speed and Air Resistance

Altitude data was given by the array of pressure sensors. Speed was then calculated based on altitude data. Because $v = \frac{dh}{dt}$, to calculate speed at a given data point i, we use the slope formula $v = (h_{i+1} - h_{i-1}) / (t_{i+1} - t_{i-1})$ to derive speed, where h_{i+1} is the data point after point i and h_{i-1} is the data point before point i.

The graph of speed vs altitude is split into two sections: ascent and descent. The data looks a little funky on the ascent and doesn't fit well to a curve.

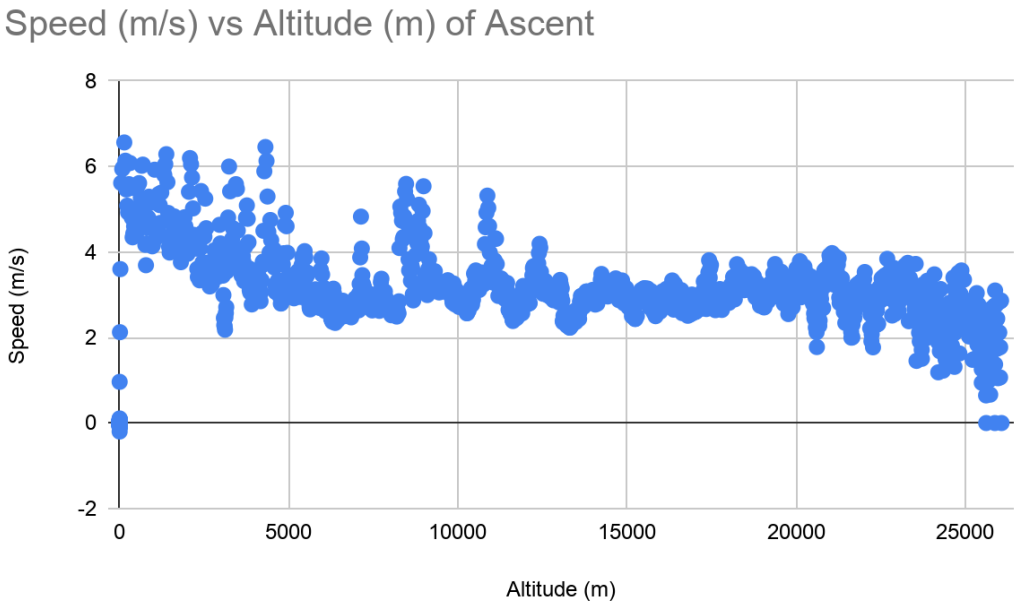


Figure 5.4.1: Graph of Speed (m/s) vs Altitude (m) on the Ascent

However, the descent data works reasonably well with an exponential fit, which is what we would expect. On the descent, our balloon popped, so we essentially recorded a graph of a free-falling payload. Near the right side of the graph (top of the balloon's flight), we see it

become more scattered and noisy, which could be due to chaotic moments after the balloon burst. An exponential trend makes sense: as the payload falls in altitude, it enters denser air exponentially, and as air gets denser, the velocity decreases exponentially.

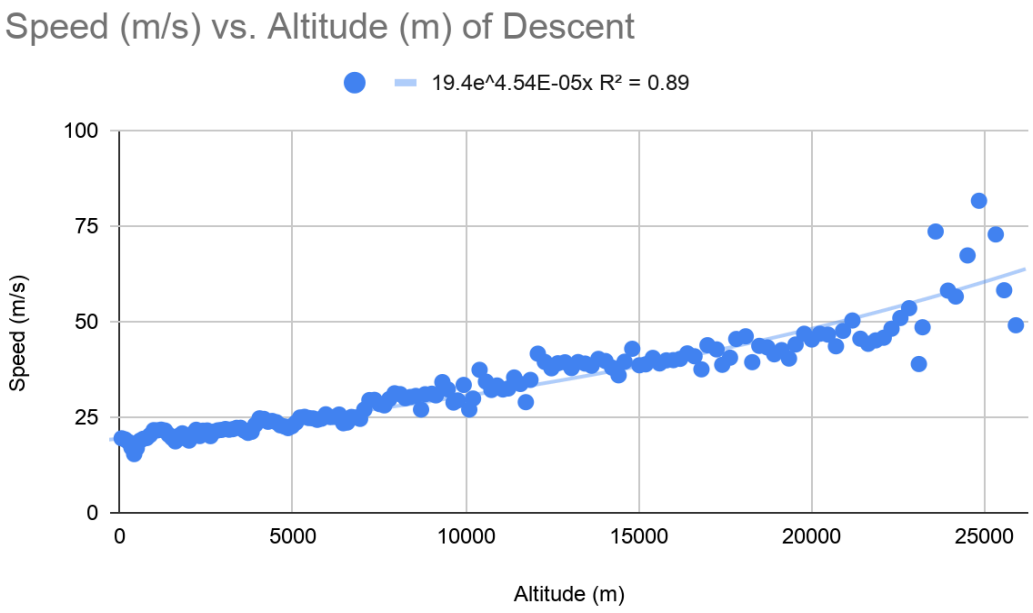


Figure 5.4.2: Graph of Speed (m/s) vs Altitude (m) on the Decent

5.5 Radioactivity vs Altitude

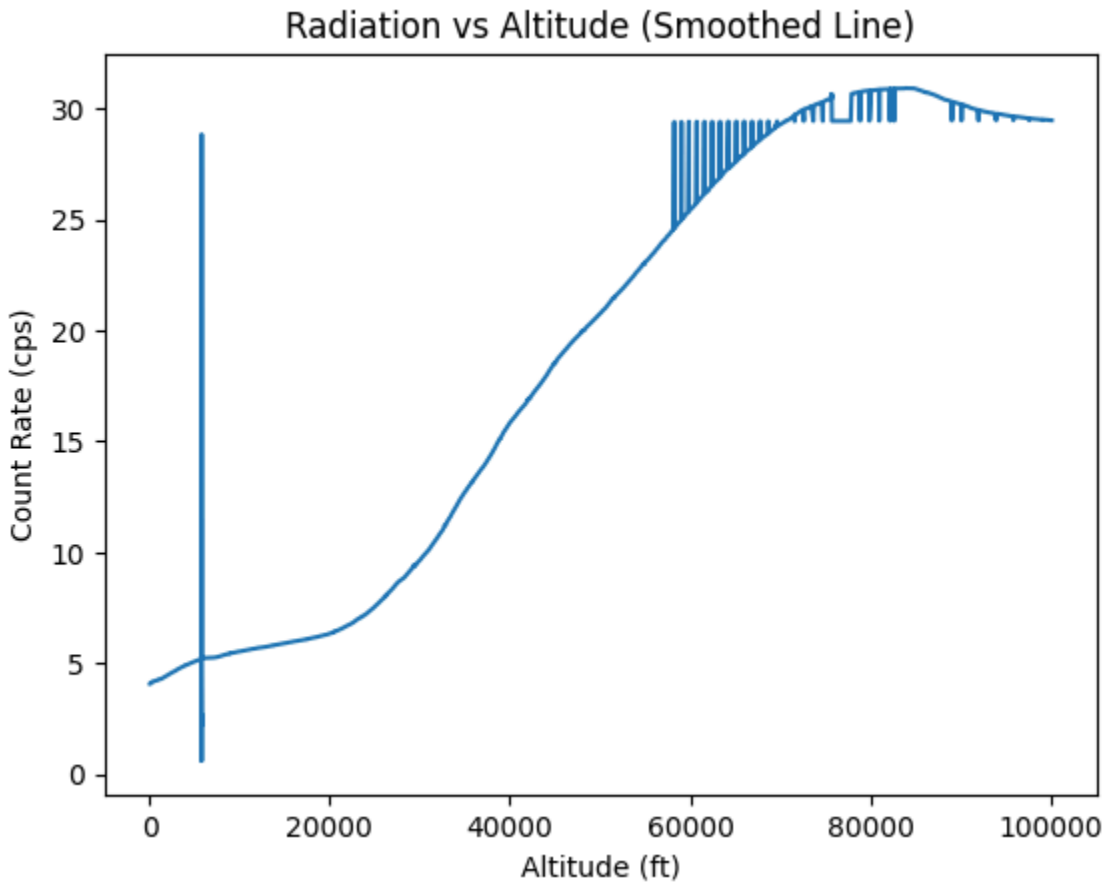


Figure 5.5.1: Radiation vs. GPS Altitude during Ascent and Descent

This graph represents the change in radiation count rate (counts per second) as a function of altitude as the balloon flies through the Earth's atmosphere. I created this graph by combining CSV data from altitude vs. time with the data collected for radiation vs time, and graphing the resulting data file. As can be seen by a quick glance at the graph above, altitude and radiation are clearly correlated. Radiation increases smoothly as altitude increases, and once the altitude gets to about 80,000 ft, it begins to level out and slowly decline. This behavior aligns closely with what would be expected theoretically.

As the weather balloon ascends, the atmosphere becomes less dense. This causes the amount of shielding from high-energy cosmic rays to be reduced, and the amount of radiation detected to increase. Specifically, more secondary particles reach the detector. Secondary particles are produced when cosmic rays collide with atmospheric nuclei. In the graph, this can be seen as the count rate rises from about 5 cps all the way to 30 cps, which is exactly what we would have expected.

Another key feature of the graph is something called the Pfotzer maximum, which is where the intensity of secondary cosmic radiation peaks. On this graph, the Pfotzer maximum occurs at approximately 80,000 feet. This altitude is where the particle production from these cosmic ray interactions are maximized, but the atmospheric absorption has not yet significantly reduced the number of detectable particles. At altitudes higher than this region, the count rate begins to level off and decline, which is also expected because of the fewer atmospheric interactions.

These radiation measurements were obtained using the Radiacode 103 detector, which is a scintillation Geiger counter (different than some other detectors). Instead of using gas that is ionized, the Radiacode contains a scintillator crystal that interacts with the incoming radiation. This produces small flashes of light which are then converted to electrical pulses and eventually contribute to the count rate. Because the detector is sensitive to these individual particle interactions, it can capture both general radiation trends and occasional high-energy events.

The data also originally included a large spike in count rate to about 590 cps. When looking at the timestamp of this spike, it was determined to correspond with the moment that the payload hit the ground. Due to the high velocity, it most likely caused the detector to malfunction. This spike was excluded from the main analysis to better reflect the true atmospheric trends.

Some sources of error in the data could include the Radiacode being inside a styrofoam box, environmental factors such as temperature changes, and variation in the orientation of the detector throughout the flight. However, despite these sources of uncertainty, the data procured does clearly illustrate a relationship between radiation and altitude that aligns well with theoretical predictions, especially with the Pfotzer maximum.

Conclusion and Code/Technical Appendix

6.1 Final Conclusion

The Balloon Launch Project allowed for team TOWEL to gain incredible insight into the intricacies of the atmosphere. This weather balloon project gave all participants a chance to experience a real-life experiment from start to finish. Everyone was able to gain practical experience and preparation for future endeavors. Most importantly, the team built a great work ethic and cooperated well with each other. The project taught team members how to design and build sensor systems from scratch, work within FAA regulations, troubleshoot hardware failures under time pressure, and analyze large datasets after the fact. Beyond the technical skills, the team learned how to adapt when things went wrong, whether that was scrapping the MS5803 sensor days before launch, discovering the thermistors were miscalibrated only after reviewing the flight data, or dealing with a parachute that never deployed. All of these problems can't be learned from a textbook, and being forced to solve them in real time made the project one of the most valuable learning experiences any of the team members had been a part of.

The data collected also yielded incredible results. The team extrapolated that pressure decreases exponentially in accordance with the standard ISA models, that the air is coldest on the border between the stratosphere and troposphere, that the rate of descent scales inversely with air resistance, and that the inside temperature is reflective of the outside but acclimates at a much slower rate. All of these findings closely matched the predictions made before the flight, confirming that the sensors and payload design performed as intended despite the calibration issues and hardware failures encountered along the way.

6.2 Code Overview

This payload had two separate microcontroller systems on board. The primary flight computer was the Arduino Uno R4 WiFi, and the Teensy 3.5 acted as a backup logger. Having both of these systems on board created a second layer of redundancy and ensured data collection just in case the main system failed.

The first large chunk of code controlled the Arduino Uno and was the main operating system for our balloon. The list of functions this code has is as follows: Reading data from the pressure, humidity, and temperature sensors; logging measurements to the SD card; transmitting telemetry data through the RTTY radio; turned on and off the heating pad; stored GPS data in the EEPROM for more redundancy in the case of power failure; controlled the cutdown and landing systems. This code used a millis() scheduling system. That means that instead of “waiting”one second between operations and causing the entire system to pause, the code instead checks the amount of time that has passed against a certain event, allowing everything to function simultaneously. Finally, a lot of the logic based functions in the code had precautions such as checking average values over a period of time and Arduino and SD card resets just in case something went wrong.

The Teensy 3.5 code was independent to the Arduino Uno and functioned as a backup data logger. Every five seconds, it would read in the outputs from the MS5611 and BME680 sensors and store them on a built-in SD card. It did not share any of the flight operations systems such as radio, heating pad, or cutdown control, and was only used for redundancy in the data collection process.

Bibliography

Anthropic. (2025). *Claude* (Large language model). https://claude.ai

COMSOL. (n.d.). *Wi-Fi booster Yagi-Uda antenna*. COMSOL Multiphysics. https://www.comsol.com/model/wi-fi-booster-yagiuda-antenna-99891

DomoticX. (n.d.). *MOSFET vs. transistor (BJT)*. https://domoticx.net/docs/mosfet-vs-transistor-bjt/

GeeksforGeeks. (2025, July 23). *Well-labelled diagram of atmosphere layers*. https://www.geeksforgeeks.org/biology/layers-of-the-atmosphere-diagram/

Keyestudio. (n.d.). *UNO R4 main board*. Keyestudio Wiki Documentation. https://docs.keyestudio.com/projects/UNO-R4-Main-Board/en/latest/docs/UNO%20R4%20Main%20Board.html

Menlo School. (n.d.). *ASR Balloon Project*. https://asr.menloschool.org/

National Oceanic and Atmospheric Administration. (n.d.). *Layers of the atmosphere*. NOAA JetStream. https://www.noaa.gov/jetstream/atmosphere/layers-of-atmosphere

NiceRF. (n.d.). *UWB module GPS*. NiceRF Wireless Technology. https://www.nicerf.com/news/uwb-module-gps.html

SchoolWorkHelper. (n.d.). *Latitude and longitude lines on the Earth*. https://schoolworkhelper.net/latitude-and-longitude-lines-on-the-earth/

Semiconductor For You. (2022, August 1). *Sensor calibration: Working & its applications*. https://www.semiconductorforu.com/sensor-calibration-working-its-applications/